



A PROJECT REPORT ON

**”TRINETRA: AI-POWERED CROWD MANAGEMENT AND
PUBLIC SAFETY SYSTEM”**

**SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE**

**IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)**

SUBMITTED BY

SHREYASH RAMESH BHOR	EXAM SEAT NO: B400690299
KRISHNA ANIL DHOTRE	EXAM SEAT NO: B400690314
JAY MACHINDRA GOSAVI	EXAM SEAT NO: B400690325
OM SANJAY KAPSE	EXAM SEAT NO: B400690337

**DEPARTMENT OF COMPUTER ENGINEERING
GES’S R.H. SAPAT COE MANAGEMENT STUDIES AND
RESEARCH**

**PRIN T.A. KULKARNI VIDYANAGAR, NASHIK - 422005
SAVITRIBAI PHULE PUNE UNIVERSITY
2025 -2026**



CERTIFICATE

This is to certify that the project report entitles
TRINETRA: AI-POWERED CROWD MANAGEMENT AND
PUBLIC SAFETY SYSTEM

Submitted by

SHREYASH RAMESH BHOR	EXAM SEAT NO: B400690299
KRISHNA ANIL DHOTRE	EXAM SEAT NO: B400690314
JAY MACHINDRA GOSAVI	EXAM SEAT NO: B400690325
OM SANJAY KAPSE	EXAM SEAT NO: B400690337

is a bonafide student of this institute and the work has been carried out by her under the supervision of Prof. M.A.Patil and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. M. A. Patil
Project Guide
Department of Computer
Engineering

Dr. D. V. Patil
HOD
Department of Computer
Engineering

(Prof.)
(External Examiner)

Dr. S. S. Sane
Principal,
Gokhale Edu. Society's
R H Sapat COE MSR, Nashik

ACKNOWLEDGEMENT

We are profoundly grateful to Prof. M.A. Patil for her expert guidance and continuous encouragement throughout to see that this project meets its target since its beginning to its completion.

We would like to express our appreciation towards Dr.S.S. Sane Principal, GES'S R H Sapat College Of Engineering Management Studies and Research. Dr. D. V. Patil, Head of Department of Computer Engineering and Prof. M.A. Patil Project Coordinator whose invaluable guidance supported us in completing this project.

At last, we must express our sincere heartfelt gratitude to all the staff members of the Computer Engineering Department who helped us directly or indirectly during this course of work.

SHREYASH RAMESH BHOR

KRISHNA ANIL DHOTRE

JAY MACHINDRA GOSAVI

OM SANJAY KAPSE

ABSTRACT

In today's world, managing large-scale public gatherings such as the Kumbh Mela, political rallies, concerts, and sports events has become increasingly challenging due to rapid population growth and safety concerns. Traditional crowd management methods depend heavily on manual surveillance and human judgment, which are often insufficient to handle dynamic and unpredictable crowd behavior. This lack of automation and real-time analysis can lead to severe accidents, overcrowding, or communication delays between authorities and on-ground staff.

To overcome these limitations, this project proposes an AI-Powered Crowd Management and Public Safety System that utilizes Artificial Intelligence (AI), Computer Vision, and Machine Learning to monitor crowd movement in real time. The system processes live video feeds from CCTV cameras to detect crowd density, and automatically send alerts to control centers and volunteers. It also integrates a web-based dashboard to visualize live heatmaps, and display environmental alerts using APIs weather forecasting services.

The proposed solution follows the Waterfall Model for systematic software development — progressing through requirement analysis, system design, implementation, testing, and deployment phases. This structured approach ensures accuracy, reliability, and maintainability of the system.

The project provides a smart, scalable, and automated solution for crowd management that enhances public safety, reduces manual effort, and improves decision making during large-scale gatherings. By leveraging AI-driven analytics and realtime visualization, it contributes to creating safer and more efficient public environments for future smart cities.

Contents

1	INTRODUCTION	1
1.1	Introduction	1
1.2	Objectives	2
1.3	Aim	2
1.4	Problem Statement	2
1.4.1	Implemented System	2
2	LITERATURE SURVEY	4
2.1	Literature Survey	4
3	SOFTWARE AND HARDWARE REQUIREMENTS SPECIFICATION	6
3.1	Introduction	6
3.1.1	Project Scope	7
3.1.2	User Classes and Characteristics	7
3.1.3	Assumptions	7
3.2	Functional Requirements	8
3.2.1	Functional Requirement 1: Real-Time Crowd Density Monitoring	8
3.2.2	Functional Requirement 2: Alert and Notification System	8
3.2.3	Functional Requirement 3: Heatmap Visualization	9
3.2.4	Functional Requirement 4: Missing Person Management	9
3.2.5	Functional Requirement 5: Weather Forecast Integration	9
3.3	System Requirements	10
3.3.1	Database Requirements	10
3.3.2	Software Requirements (Platform Choice)	10
3.3.3	Hardware Requirements	10
3.3.4	System Security Requirements	10
3.4	External Interface Requirements	11
3.4.1	Hardware Interfaces	11
3.4.2	Software Interfaces	11
3.4.3	Communication Interfaces	11
3.5	Non-Functional Requirements	12
3.5.1	Performance Requirements	12
3.5.2	Safety Requirements	12
3.5.3	Security Requirements	12
3.5.4	Software Quality Attributes	12

4	SYSTEM ANALYSIS	13
4.1	Analysis Models: SDLC Model Used	13
4.2	System Implementation Plan	14
4.2.1	Phase 1: Requirement Analysis and Setup	14
4.2.2	Phase 2: AI Model Development and Integration	14
4.2.3	Phase 3: Frontend and Backend Development	14
4.2.4	Phase 4: Testing and Validation	14
4.2.5	Phase 5: Deployment and Maintenance	15
4.3	Project Estimation	15
4.3.1	Effort Estimation	15
4.3.2	Time Estimation	16
4.3.3	Resource Estimation	16
4.3.4	Cost Estimation	16
4.4	Project Work Breakdown Structure	17
4.4.1	Phase 1: Requirement Analysis	17
4.4.2	Phase 2: System Design	17
4.4.3	Phase 3: Frontend Development	17
4.4.4	Phase 4: Backend Development	17
4.4.5	Phase 5: AI Model Development	18
4.4.6	Phase 6: Testing and Validation	18
4.4.7	Phase 7: Deployment and Maintenance	18
5	SYSTEM DESIGN	19
5.1	Use Case Diagram	19
5.2	Data Flow Diagram Level-0	20
5.3	Data Flow Diagram Level-1	21
5.4	Data Flow Diagram Level-2	22
5.5	Entity Relationship Diagram	24
5.6	Class Diagram	25
5.7	Component Diagram	26
5.8	State Machine Diagram	27
5.9	Sequence Diagram	28
5.10	Deployment Diagram	30
6	SYSTEM ARCHITECTURE	31
6.1	System Architecture	31
6.1.1	Data Acquisition and Preprocessing Layer	32
6.1.2	Feature Extraction and Transformation Layer	33
6.1.3	AI-Based Crowd Density Estimation Layer	34
6.1.4	Model Training Layer	35
6.1.5	Inference and Model Evaluation Layer	36
6.1.6	Backend Integration and API Communication Layer	37
6.1.7	Database Management Layer	38
6.1.8	Frontend Visualization Layer	39

7	WORKING METHODOLOGY	40
7.1	Methodological Strategy	40
7.1.1	Data Acquisition and Preprocessing Layer	40
7.1.2	Feature Extraction and Transformation Layer	40
7.1.3	AI-Based Crowd Density Estimation Layer	41
7.1.4	Model Training Layer	41
7.1.5	Inference and Model Evaluation Layer	41
7.1.6	Backend Integration and API Communication Layer	42
7.1.7	Database Management Layer	42
7.1.8	Frontend Visualization Layer	42
7.2	Project Modules	44
7.2.1	Interface	44
7.2.2	Code Snippets	45
8	TESTING AND RESULTS	46
8.1	Testing	46
8.2	Testing Strategies	47
8.2.1	Functional Testing	47
8.2.2	Integration Testing	47
8.2.3	Performance Testing	47
8.2.4	Security Testing	47
8.3	Results	47
8.4	Performance Evaluation Metrics	48
8.4.1	Mean Absolute Error (MAE):	48
8.4.2	Mean Absolute Error (MAE):	48
8.4.3	Root Mean Squared Error (RMSE):	48
8.4.4	Prediction Accuracy within Error Thresholds:	48
8.4.5	Mean Prediction Accuracy:	49
8.4.6	Crowd Density Heatmap Visualization:	49
8.4.7	Actual vs Predicted Crowd Count Comparison:	49
8.4.8	Confusion Matrix:	49
9	RESULTS	50
9.1	Output	50
9.1.1	Crowd Density Heatmap Result:	50
9.1.2	Confusion Matrix Result:	51
9.1.3	Advanced Model Metrics Result:	52
10	CONCLUSION & FUTURE SCOPE	53
10.1	Conclusion	53
10.2	Future Scope	54
	REFERENCES	55
10.3	References	55

List of Figures

5.1	Use Case Diagram	19
5.2	DFD level:0 Diagram	20
5.3	DFD level:1 Diagram	21
5.4	DFD level:2 Diagram	23
5.5	Entity Relationship Diagram	24
5.6	Class Diagram	25
5.7	Component Diagram	26
5.8	State Machine Diagram	27
5.9	Sequence Diagram	29
5.10	Deployment Diagram	30
6.1	System Architecture Diagram	31
7.1	User Interface	44
7.2	Implementation Code	45
7.3	Implementation Code	45
9.1	Density Heatmap	50
9.2	Confusion Matrix	51
9.3	Evaluation Metrics	52

List of Tables

3.1	Software Requirements	10
3.2	Hardware Requirements	10
4.1	Project Time Estimation	16

Chapter 1

INTRODUCTION

1.1 Introduction

Crowd management has become a major challenge in today's rapidly urbanizing and densely populated world. Events such as the Kumbh Mela, political rallies, sports tournaments, and concerts attract massive numbers of people within limited spaces, creating potential risks related to overcrowding, accidents, and communication delays. Traditional crowd control methods depend largely on human supervision and manual reporting, which are not only time-consuming but also prone to human error. The absence of automated monitoring systems makes it difficult to respond quickly to emergencies or predict possible hazards in dynamic crowd situations.

The objectives of proposed project, AI-Powered Crowd Management and Public Safety System, is to address these issues through a smart, data-driven, and real-time solution. By leveraging the power of Artificial Intelligence (AI), Computer Vision, and Machine Learning (ML), the system continuously analyzes live video feeds from surveillance cameras to estimate crowd density, detect unusual patterns, and predict possible crowd surges. The processed information is then displayed on a web-based dashboard that provides visual heatmaps, safe route suggestions, and automated alerts for the authorities. This enables faster decision-making, better coordination among officials, and improved safety for the public.

The system's architecture integrates various technologies such as CNN-based image analysis, Weather APIs, and weather forecasting tools to deliver a comprehensive situational overview. It also includes a missing person management feature, where volunteers can upload information to assist authorities in locating lost individuals. This blend of AI-powered detection, real-time communication, and visualization ensures effective crowd management even under high-pressure scenarios.

The AI-Powered Crowd Management and Public Safety System represents a significant step toward building intelligent, technology-driven public safety solutions. It demonstrates how AI can enhance surveillance, forecasting, and coordination in real-world scenarios, minimizing human error and enabling safer management of large gatherings.

1.2 Objectives

The main objective of the project “AI-Powered Crowd Management and Public Safety System” is to design and develop an intelligent, real-time system that ensures public safety during large gatherings using Artificial Intelligence and Computer Vision.

The specific objectives are:

- To implement AI-based crowd detection using live CCTV video feeds for realtime density estimation.
- To provide a web-based dashboard that displays live heatmaps and alerts.
- To generate automatic alerts and notifications in case of overcrowding or emergency situations.
- To provide local live weather and weather forecast for environmental awareness

1.3 Aim

1. To design and develop an AI-powered crowd management system for real-time crowd monitoring and public safety.
2. To implement crowd detection and density estimation using live CCTV video feeds with the help of Computer Vision and Machine Learning techniques.
3. To generate automatic alerts during overcrowding or emergency situations.
4. To provide a web-based dashboard for live heatmap visualization, and crowd analytics.
5. To assist authorities and volunteers in missing person management by providing realtime reports and updates.

1.4 Problem Statement

To design and develop an AI-powered, web-based crowd management and public safety system to detect real-time crowd density, assist in missing person management, visualize heatmaps, and manage weather forecast alerts efficiently during large-scale gatherings like the Kumbh Mela, using computer vision (CNN), machine learning models, weather forecasting APIs.

1.4.1 Implemented System

The implemented crowd management system uses machine learning technology to monitor real-time crowd activity and efficiently manage large public gatherings. The system collects data through surveillance systems and continuously monitors crowd movement while estimating crowd density in real time using heatmaps and machine learning models. The implemented machine learning algorithms help identify overcrowded areas and display them clearly on the density map for better monitoring and analysis.

The system includes a live video visualization module that enables authorities to monitor high-density zones and take immediate action whenever required. It also provides automatic emergency alerts to security personnel and event organizers for effective crowd handling and management. Communication between volunteers, security staff, and authorities is improved through instant notifications and a centralized monitoring dashboard.

Additional features such as weather alert integration and missing person management further enhance public safety during large-scale public events and gatherings. By automating surveillance, decision-making processes, and coordination with on-ground authorities, the implemented system reduces manual effort, improves risk management, and increases the overall efficiency and safety of crowd management operations.

Chapter 2

LITERATURE SURVEY

2.1 Literature Survey

In the paper [1], the authors discuss the application of Artificial Intelligence technologies in managing large-scale religious gatherings such as Hajj and Umrah. The study explains that traditional crowd management methods are not sufficient for handling millions of people in highly crowded environments. To improve safety and coordination, the research explores the use of AI technologies such as machine learning, computer vision, facial recognition, IoT devices, and predictive analytics for crowd monitoring and emergency response.

The paper highlights that AI-based systems can monitor crowd movement in real time, identify overcrowded areas, and predict potential risks before accidents occur. The study also explains how AI technologies help authorities improve transportation management, emergency communication, and missing person identification during large religious gatherings. Furthermore, the research emphasizes that intelligent systems can optimize crowd flow and improve public safety through automated decision-making and real-time analysis. Although the research identifies challenges related to privacy, ethics, and cultural sensitivity, it concludes that AI technologies can significantly enhance crowd management efficiency and public safety during massive gatherings.

In the paper [2], the authors present an AI-based crowd behavior analysis system using dynamic pedestrian-agent models to study and predict crowd movement patterns. The study explains that traditional crowd monitoring systems are limited because they rely heavily on manual observation and fixed surveillance methods. To improve crowd management, the proposed system models each pedestrian as an intelligent agent capable of adapting to surrounding crowd conditions and environmental changes. The system analyzes real-time crowd interactions and predicts congestion, abnormal movement, and possible crowd surges before dangerous situations occur.

The research demonstrates that AI-driven simulation techniques can significantly improve public safety during large-scale events such as festivals, sports events, and religious gatherings. By using machine learning and crowd behavior analysis, the model can support emergency planning, optimize crowd flow, and reduce the risk of

accidents or stampedes. The study also highlights that the proposed approach performs effectively even in high-density crowd environments where traditional object detection methods face difficulties due to occlusion and overlapping individuals. Although the model requires high computational resources and large training datasets, the paper concludes that intelligent pedestrian-agent systems provide an effective and promising solution for future AI-powered crowd management applications.

In the paper [3], the authors provide a comprehensive survey of deep learning techniques used for crowd counting and density estimation in computer vision systems. The study explains that traditional crowd counting methods often fail in highly crowded environments due to challenges such as occlusion, scale variation, poor lighting, and overlapping individuals. To overcome these issues, the paper discusses the use of Convolutional Neural Networks (CNNs) and advanced deep learning architectures that can automatically learn crowd features from images and video frames.

The study highlights that density map estimation has become one of the most effective approaches for analyzing dense crowds. Instead of detecting every individual person separately, deep learning models generate density maps that estimate the concentration of people in different regions of an image. The paper also compares various CNN-based models, including multi-column networks and scale-aware architectures, which improve crowd counting accuracy under varying crowd densities and perspectives. Furthermore, the research emphasizes that deep learning techniques can support real-time crowd management systems by providing accurate crowd statistics, congestion analysis, and anomaly detection capabilities. Although these models require large datasets and high computational resources, the paper concludes that deep learning has significantly improved the efficiency and reliability of intelligent crowd monitoring systems.

In the paper [4], the authors propose a smart vision-based public safety system that combines deep learning and edge computing for real-time crowd monitoring and anomaly detection. The study explains that traditional surveillance systems often suffer from high latency and internet dependency, making them less effective for real-time emergency response. To overcome these limitations, the proposed system uses a hybrid deep learning architecture integrating YOLOv8 object detection with ConvLSTM models for analyzing crowd behavior and detecting abnormal activities.

The research demonstrates that the proposed model achieved high anomaly detection accuracy while maintaining real-time processing performance. By using edge computing, the system processes surveillance data locally near the cameras instead of depending completely on cloud servers, reducing response delays and improving reliability. The study also highlights that combining spatial and temporal crowd analysis improves the accuracy of detecting suspicious activities and overcrowded areas. Furthermore, the paper concludes that intelligent edge-based surveillance systems can play an important role in future smart city and public safety applications by enabling faster, scalable, and more efficient real-time crowd monitoring solutions.

Chapter 3

SOFTWARE AND HARDWARE REQUIREMENTS SPECIFICATION

3.1 Introduction

This Software and Hardware Requirements Specification (SHRS) defines the functional and non-functional requirements of the proposed project “TRINETRA – AI Powered Crowd Management and Public Safety System”. The sections describes the software components, hardware requirements, system functionalities, and technical constraints necessary for the successful development and deployment of the system. The implemented system is designed to provide real-time crowd monitoring, crowd density estimation, emergency alert generation, and public safety management using Artificial Intelligence (AI), Computer Vision, and Machine Learning techniques. The system analyzes CCTV video frames and detect overcrowded areas efficiently by generating density based heatmaps.

The specification also describes the technologies, tools, APIs, databases, and communication interfaces used in the system. It includes details about frontend and backend requirements, database management, external services, and system infrastructure required for smooth operation.

The system aims to improve public safety during large-scale gatherings such as the Kumbh Mela, concerts, political rallies, and sports events by providing automated monitoring, live heatmap visualization, weather alerts, and safe navigation recommendations.

This module serves as a guideline for developers, project guides, and users to understand the overall system requirements, design constraints, and operational capabilities of the proposed solution.

3.1.1 Project Scope

The scope of the project “TRINETRA – AI Powered Crowd Management and Public Safety System” is to develop an intelligent and automated platform for monitoring and managing large public gatherings using Artificial Intelligence (AI), Computer Vision, and Machine Learning techniques. The system analyzes CCTV video frames to detect crowd density and identify overcrowded areas in real time. It provides a web-based dashboard that displays live heatmaps, weather alerts and crowd analytics for authorities and volunteers. The project also includes automatic alert generation during emergency situations, missing person management and density maps. Integration with services such as OpenWeatherMap API that improves real-time communication and environmental awareness. OpenStreetMap API helps display the crowd density on map. The implemented system can be used in large gatherings such as the Kumbh Mela, political rallies, concerts, festivals, and sports tournaments. The main objective of the system is to improve public safety, reduce manual effort, and support faster decision-making during large-scale public gatherings.

3.1.2 User Classes and Characteristics

The implemented system consists of different types of users who interact with the application according to their roles and responsibilities. Each user class has specific access permissions and functionalities within the system.

- **Admin:** The admin is responsible for monitoring crowd conditions, managing system activities, viewing live heatmaps, handling alerts, and managing volunteers and missing person reports. Admin users have full access to the dashboard and system controls.
- **Volunteer:** Volunteers assist authorities during large public gatherings. They can report incidents, post alerts, upload missing person information, and communicate with administrators through the system.
- **General User:** General users can view crowd-related information such as density level map, live alerts, weather updates and forecast.

3.1.3 Assumptions

- The system assumes that CCTV cameras and surveillance devices are properly installed and continuously available during public events.
- Stable internet connectivity is required for real-time video streaming, dashboard updates, and communication between system modules.
- It is assumed that administrators and volunteers have basic knowledge of using web-based applications.
- The system assumes that event authorities will respond promptly to generated alerts and notifications.

- It is assumed that crowd zones and alert thresholds are correctly configured by the admin before the event begins.
- The system assumes that volunteers are present on the ground and can submit field reports and missing person updates in a timely manner.
- It is assumed that users have access to a smartphone or desktop browser to view live heatmaps, alerts, and safe navigation routes.
- The system assumes that environmental conditions such as lighting and camera placement are adequate for accurate crowd detection.

3.2 Functional Requirements

3.2.1 Functional Requirement 1: Real-Time Crowd Density Monitoring

- The system shall receive processed video frames from surveillance systems for crowd analysis.
- The system shall use Computer Vision and CNN-based models to detect people present in each frame.
- The system shall calculate crowd density for different monitored zones in real time.
- The system shall identify high-density and overcrowded areas based on pre-defined threshold values.
- The system shall continuously update crowd information on the web dashboard without requiring manual refresh.

3.2.2 Functional Requirement 2: Alert and Notification System

- The system shall generate automatic alerts whenever crowd density exceeds predefined safety limits.
- The system shall notify administrators and volunteers through dashboard notifications and real-time updates.
- The system shall represent crowd conditions using color indicators such as Green (Safe), Yellow (Moderate), and Red (Overcrowded).
- The system shall maintain logs of alerts and notifications in the database.
- The system shall support real-time communication using WebSocket or Socket.io technology.

3.2.3 Functional Requirement 3: Heatmap Visualization

- The system shall display real-time crowd heatmaps on the dashboard using Google Maps API.
- The system shall visualize crowded and safe zones using different color-coded regions.
- The system shall dynamically update heatmaps according to live crowd density information.
- The system shall provide location-based visualization for effective crowd monitoring and management.
- The system shall display real-time crowd status information for administrators, volunteers, and users.

3.2.4 Functional Requirement 4: Missing Person Management

- The system shall allow volunteers to upload lost or found person information along date of birth and location details.
- The system shall securely store missing person records in the database.
- The system shall allow administrators and volunteers to monitor and manage missing person cases through the dashboard.
- The system shall maintain status updates for resolved and unresolved missing person reports.

3.2.5 Functional Requirement 5: Weather Forecast Integration

- The system shall integrate Weather Forecast APIs to fetch real-time weather information.
- The system shall display environmental conditions such as temperature, humidity, rainfall, and weather warnings on the dashboard.
- The system shall generate alerts during severe weather conditions affecting crowd safety.
- The system shall assist administrators in decision-making using weather-related safety information.
- The system shall forecast next 5 days weather for the current location.

3.3 System Requirements

3.3.1 Database Requirements

- The database should be MongoDB v8.3.2 (latest)
- The database should support real-time data updates and secure storage mechanisms.

3.3.2 Software Requirements (Platform Choice)

Component	Specification
Operating System	Windows 10 or Linux
Frontend Technologies	Next.js v15.5.2, React.js v19.0.1
Backend Technology	Node.js, FastAPI
Database	MongoDB v8.3.2
APIs	OpenWeather Map API, OpenStreetMap API
Browser Support	Google Chrome, Microsoft Edge, Mozilla Firefox

Table 3.1: Software Requirements

3.3.3 Hardware Requirements

Hardware Component	Specification
Processor	Intel Core i5 or higher
RAM	Minimum 8 GB
Storage	Minimum 256 GB SSD

Table 3.2: Hardware Requirements

3.3.4 System Security Requirements

- The system should provide secure login authentication for administrators and volunteers.
- Sensitive data should be protected from unauthorized access.
- Secure APIs and encrypted communication methods should be used for data transfer.

3.4 External Interface Requirements

3.4.1 Hardware Interfaces

- CCTV or drone cameras for live video input.
- GPU-enabled computing system for AI processing.
- Server or cloud hosting for backend operations.
- Edge computing devices for local AI processing.
- Dedicated GPU servers for deep learning computations.

3.4.2 Software Interfaces

- AI libraries: OpenCV, TensorFlow.
- Database: MongoDB.
- Backend frameworks: Node.js (JavaScript).
- APIs: OpenStreetMap API, Openweathermap API.
- Polling for real-time bidirectional communication.
- JWT authentication for secure login sessions.
- Bcrypt for password hashing and encryption.
- RESTful APIs for frontend-backend integration.

3.4.3 Communication Interfaces

- Communication between frontend and backend using REST APIs.
- Real-time data transfer via Polling.
- Cloud-based data synchronization using MongoDB Database.
- Real-time notifications for emergency alerts.
- Secure HTTPS-based communication protocols.
- Polling communication for continuous live updates.
- Cloud synchronization between distributed monitoring centers.

3.5 Non-Functional Requirements

3.5.1 Performance Requirements

- The system must process each video frame within 1–2 seconds.
- Dashboard updates should occur in real time without page reloads.
- Must support at least 10 camera feeds simultaneously.
- The alert notification system should respond within a few seconds after detecting overcrowding.
- The AI model should maintain high accuracy even in dense crowd conditions.

3.5.2 Safety Requirements

- The system must generate warnings and alerts in case of high crowd density.
- Prevent data loss during unexpected shutdowns or disconnections.
- Emergency alerts should be immediately visible to admin, volunteers and general user.
- The system should forecast future weather.

3.5.3 Security Requirements

- Role-based access (Admin, Volunteer, General User).
- Passwords stored securely using hashing and encryption.
- Secure communication via HTTPS and JWT authentication.
- Unauthorized users should not be allowed to access admin functionalities.

3.5.4 Software Quality Attributes

- Usability: Simple, user-friendly UI with clear color-coded indicators.
- Reliability: Continuous operation during events without crashes.
- Scalability: Capable of handling more cameras or users as required.
- Maintainability: Modular code structure for easy updates and debugging.
- Portability: Compatible with multiple browsers and cloud platforms.

Chapter 4

SYSTEM ANALYSIS

4.1 Analysis Models: SDLC Model Used

The proposed project “TRINETRA – AI Powered Crowd Management and Public Safety System” follows the Waterfall Model of the Software Development Life Cycle (SDLC). The Waterfall Model provides a structured and sequential approach for developing the system through multiple phases such as requirement analysis, system design, implementation, testing, deployment, and maintenance. This model helps in systematic planning, proper documentation, and efficient project management throughout the development process.

In the Requirement Analysis phase, the functional and non-functional requirements of the system are identified and documented. The System Design phase focuses on designing the system architecture, database structure, user interfaces, APIs, and AI integration modules. The Implementation phase involves developing the frontend dashboard, backend services, database connectivity, and AI-based crowd density detection modules using the required technologies.

The Testing phase verifies the functionality, performance, reliability, and security of the system. Crowd density detection accuracy, alert generation, heatmap visualization, and database operations are tested to ensure proper system performance. After successful testing, the system is deployed for real-time crowd monitoring and public safety management. The Maintenance phase includes bug fixing, performance optimization, and future system enhancements.

- Requirement Analysis
- System Design
- Implementation
- Testing
- Deployment
- Maintenance

4.2 System Implementation Plan

The implementation plan of the proposed project “TRINETRA – AI Powered Crowd Management and Public Safety System” is divided into different phases for systematic development and deployment of the system. Each phase focuses on frontend development, backend integration, AI-based crowd density analysis, testing, and deployment.

4.2.1 Phase 1: Requirement Analysis and Setup

- Requirement analysis and project planning.
- Setup of development environment and database.
- Configuration of frontend, backend, and AI development tools.
- Collection and preparation of crowd image datasets for testing and model evaluation.

4.2.2 Phase 2: AI Model Development and Integration

- Development of crowd detection and density estimation models.
- Integration of OpenCV and TensorFlow libraries for image processing.
- Implementation of overcrowding detection mechanisms based on density thresholds.
- Integration of AI modules with the backend system.

4.2.3 Phase 3: Frontend and Backend Development

- Development of the web dashboard using Next.js and React.js.
- Implementation of heatmap visualization and live alert modules.
- Integration of weather APIs for environmental updates and alerts.
- Development of backend APIs using Node.js and FastAPI.
- Integration of MongoDB database for storing crowd data, alerts, and user information.
- Implementation of WebSocket or Socket.io for real-time dashboard updates.

4.2.4 Phase 4: Testing and Validation

- Testing of AI-based crowd density detection modules.
- Verification of dashboard functionalities and real-time updates.
- Testing of alert generation and database operations.
- Performance, reliability, and security testing of the complete system.

4.2.5 Phase 5: Deployment and Maintenance

- Deployment of the system on server infrastructure or cloud platform.
- Real-time monitoring and maintenance of system performance.
- Bug fixing, optimization, and future system improvements.

4.3 Project Estimation

Project estimation provides an approximate analysis of the time, effort, resources, and technologies required for successful development of “TRINETRA – AI Powered Crowd Management and Public Safety System”. The estimation helps in proper planning, scheduling, and management of project activities.

It also assists in identifying the overall development cost, manpower requirements, and implementation feasibility of the system. Proper estimation ensures efficient utilization of available resources and helps in minimizing project risks and delays. The estimation process further supports timely completion of the project while maintaining system quality and performance standards.

4.3.1 Effort Estimation

The development of the system requires collaborative effort in frontend development, backend integration, AI model implementation, database management, testing, and deployment.

- Frontend Development : Development of dashboard, heatmaps, alerts, authentication, and responsive user interface using Next.js and React.js.
- Backend Development : Development of APIs, real-time communication modules, database integration, and authentication using Node.js and FastAPI.
- AI Model Development : Crowd density detection and overcrowding identification using OpenCV, TensorFlow, and CNN models.
- Database Management : Designing and managing MongoDB collections for alerts, crowd logs, user information, and missing person records.
- Testing and Validation : Functional testing, integration testing, performance testing, and security testing of the complete system.
- Deployment and Maintenance : Deployment on server infrastructure and continuous maintenance of the system.

4.3.2 Time Estimation

The project development is divided into different phases with estimated duration for each activity.

Project Phase	Estimated Duration
Requirement Analysis and Planning	1 Week
System Design and Architecture	1 Week
Frontend Development	2 Weeks
Backend and Database Development	2 Weeks
AI Model Development and Integration	2 Weeks
Testing and Validation	1 Week
Deployment and Documentation	1 Week

Table 4.1: Project Time Estimation

4.3.3 Resource Estimation

- Development System with minimum Intel Core i5 processor and 8 GB RAM.
- Stable internet connection for API integration and real-time communication.
- MongoDB database for cloud-based storage and synchronization.
- OpenCV and TensorFlow libraries for AI-based crowd density analysis.
- Next.js, React.js, Node.js, and FastAPI frameworks for application development.

4.3.4 Cost Estimation

- Software tools and frameworks used in the project are open-source.
- Cloud database and hosting services may require operational cost during deployment.
- API usage charges may apply for weather services and mapping services.
- Hardware cost includes development systems and surveillance infrastructure.

4.4 Project Work Breakdown Structure

The Work Breakdown Structure (WBS) divides the project into smaller and manageable activities for systematic development and implementation of the proposed system.

4.4.1 Phase 1: Requirement Analysis

- Study of crowd management and public safety systems.
- Identification of functional and non-functional requirements.
- Analysis of system scope and user requirements.
- Preparation of Software Requirement Specification (SRS) document.

4.4.2 Phase 2: System Design

- Design of system architecture and workflow.
- Design of database schema and data flow.
- Preparation of UML diagrams such as Use Case Diagram, DFD, ER Diagram, and Sequence Diagram.
- Design of dashboard user interface and visualization modules.

4.4.3 Phase 3: Frontend Development

- Development of dashboard using Next.js and React.js.
- Implementation of authentication pages for admin and volunteers.
- Development of heatmap visualization and alert display modules.
- Integration of weather information and live dashboard updates.
- Development of responsive user interface for desktop and mobile devices.

4.4.4 Phase 4: Backend Development

- Development of REST APIs using Node.js and FastAPI.
- Integration of MongoDB database with backend services.
- Implementation of authentication and authorization mechanisms.
- Implementation of WebSocket or Socket.io for real-time communication.
- Development of modules for alert management and missing person records.

4.4.5 Phase 5: AI Model Development

- Dataset collection and preprocessing for crowd analysis.
- Development of CNN-based crowd density detection model.
- Integration of OpenCV and TensorFlow libraries.
- Implementation of overcrowding detection based on density thresholds.
- Integration of AI model with backend APIs.

4.4.6 Phase 6: Testing and Validation

- Functional testing of frontend and backend modules.
- Testing of crowd density detection accuracy.
- Verification of alert generation and heatmap updates.
- Database testing and API testing.
- Performance, reliability, and security testing.

4.4.7 Phase 7: Deployment and Maintenance

- Deployment of application on server or cloud platform.
- Configuration of database and APIs.
- Monitoring of real-time system performance.
- Bug fixing and optimization.
- Future maintenance and feature enhancement.

Chapter 5

SYSTEM DESIGN

5.1 Use Case Diagram

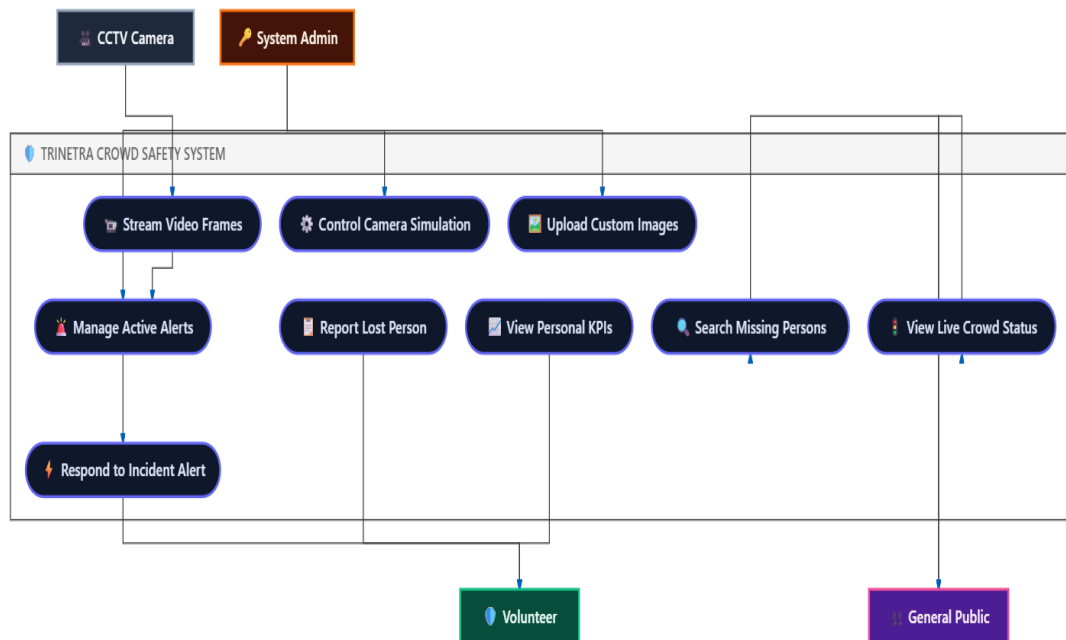


Figure 5.1: Use Case Diagram

The Use Case Diagram for the Admin in the AI-Powered Crowd Management and Public Safety System illustrates the various functionalities managed by the administrator. The Admin is responsible for monitoring live crowd data, managing volunteers, broadcasting alerts, reviewing field reports, handling missing person cases, and configuring system settings such as alert thresholds and zone priorities. This diagram highlights how the Admin interacts with different system modules to oversee operations, ensure safety, and maintain real-time control during crowd management situations.

5.2 Data Flow Diagram Level-0

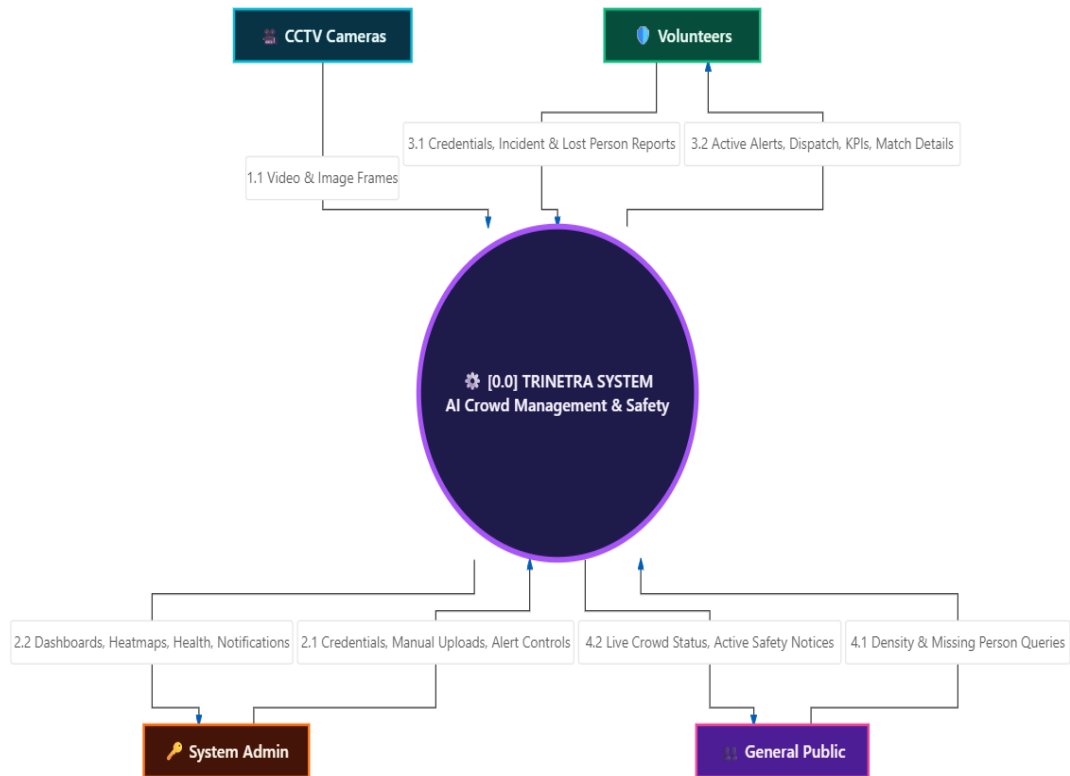


Figure 5.2: DFD level:0 Diagram

The Data Flow Diagram (DFD) Level 0, also known as the Context Diagram, represents the overall working boundary of the “TriNetra – AI Powered Crowd Management and Public Safety System” by showing the complete system as a single central process interacting with external entities. The system receives crowd images and video frames from CCTV cameras for AI-based crowd density analysis. Administrators provide system configurations and receive crowd alerts and monitoring information, while volunteers submit incident and missing person reports and receive alert notifications. General users access crowd density information, heatmaps, and safety updates through the system dashboard. The diagram provides a high-level overview of data flow between the system and all external entities without showing internal processing details.

5.3 Data Flow Diagram Level-1

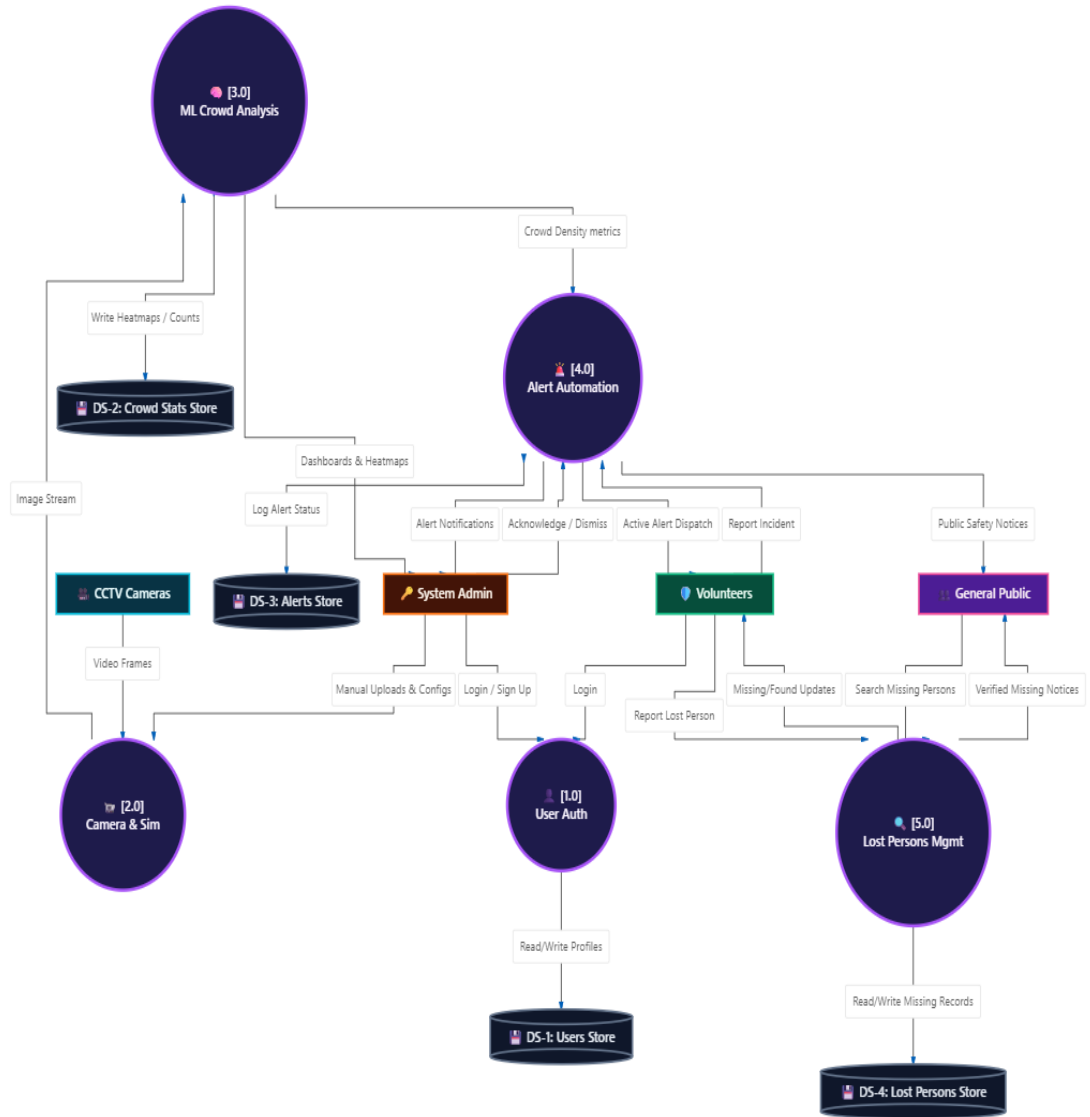


Figure 5.3: DFD level:1 Diagram

The Data Flow Diagram (DFD) Level 1 of the “TriNetra – AI Powered Crowd Management and Public Safety System” divides the main system into major functional modules such as User Authentication, Camera Processing, AI-Based Crowd Analysis, Alert Management, and Missing Person Management. Crowd images and video frames received from CCTV cameras are processed through the AI crowd analysis module for crowd density estimation and overcrowding detection. The generated crowd statistics are stored in the database and used by the alert management module to generate notifications for administrators and volunteers. The system also manages missing person records submitted by volunteers and accessed by users.

5.4 Data Flow Diagram Level-2

The DFD Level 2 of the AI-Powered Crowd Management and Public Safety System provides a detailed view of how each major process works internally. It shows the step-by-step data flow between sub-processes such as Crowd Detection and Analysis, Alert Generation, Volunteer Response Handling, and User Notification Management. The AI module analyzes live sensor data to calculate crowd density and detect potential risks. When critical levels are reached, alerts are created and sent to the Admin for approval and Volunteers for ground verification. After the issue is resolved, the system updates the database and notifies Users with safe routes and current status. This diagram illustrates the complete internal data handling and decision-making workflow of the system.

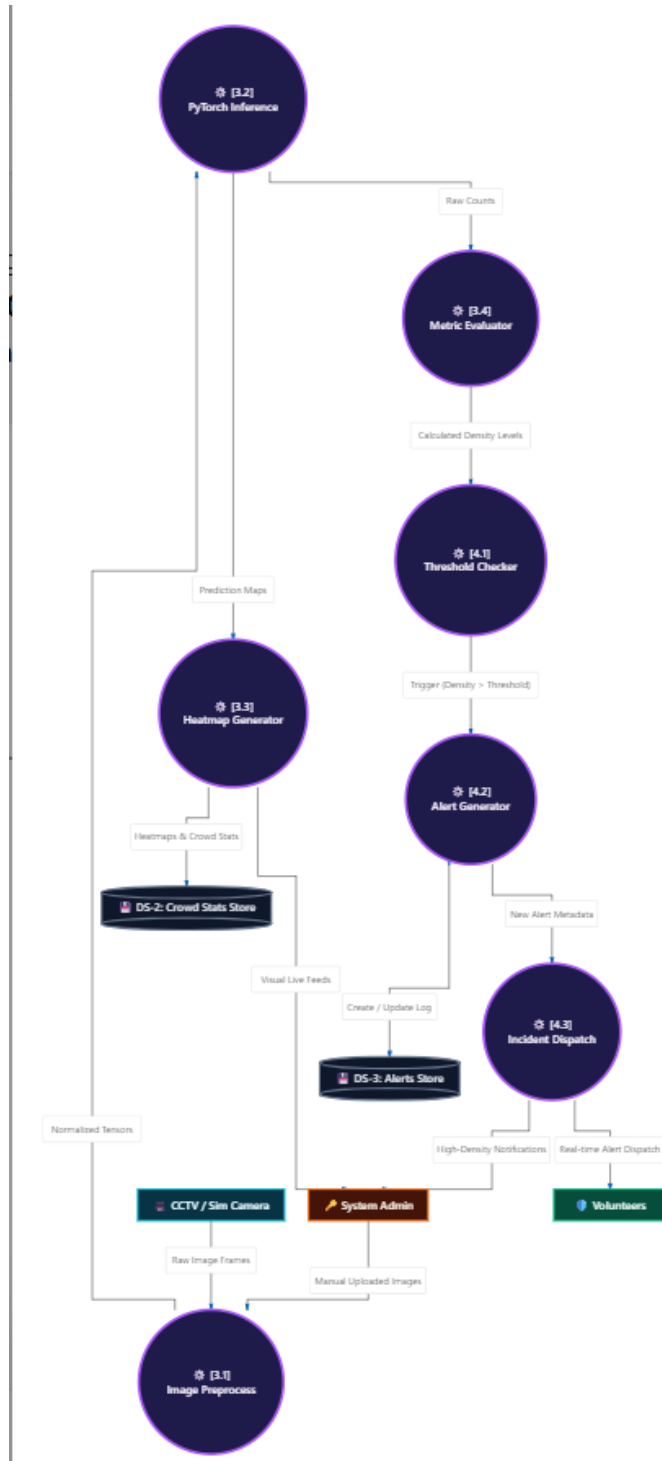


Figure 5.4: DFD level:2 Diagram

5.5 Entity Relationship Diagram

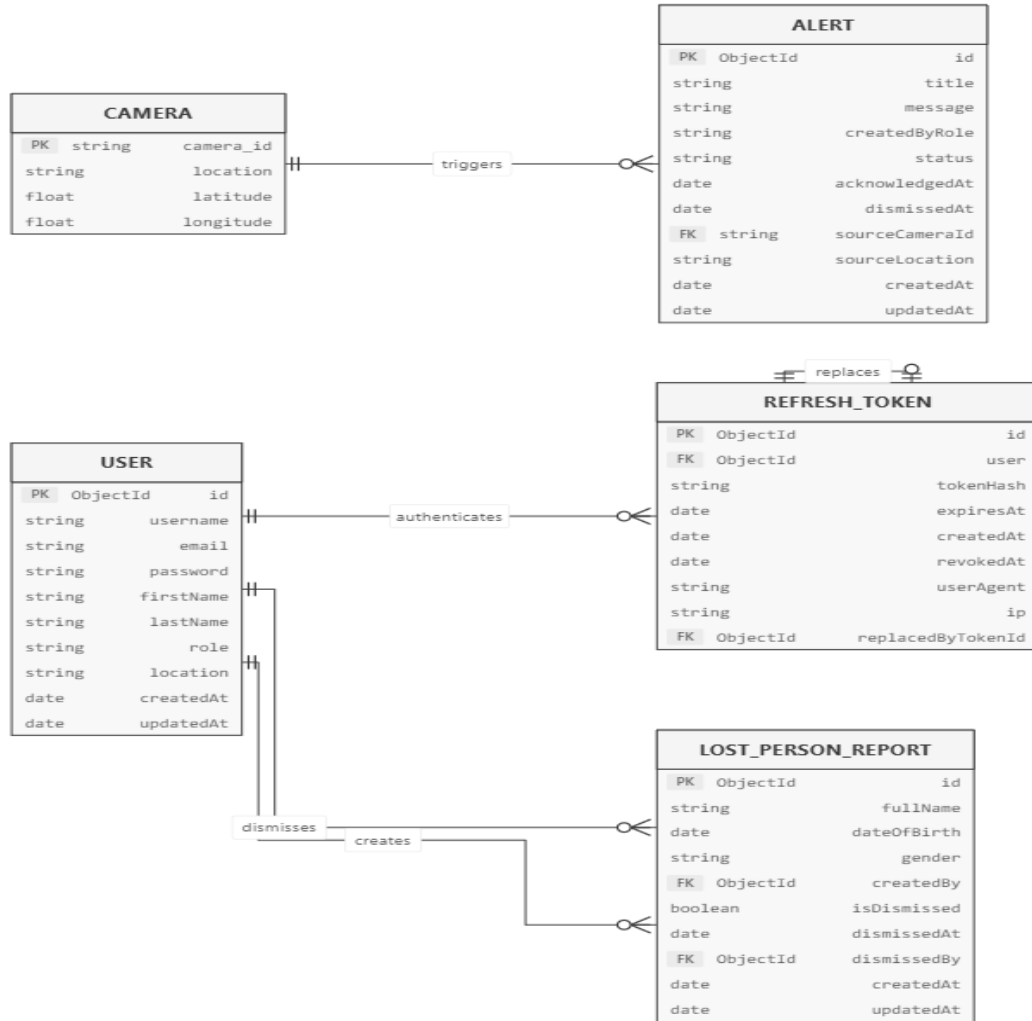


Figure 5.5: Entity Relationship Diagram

The Entity-Relationship (ER) Diagram of the “TriNetra – AI Powered Crowd Management and Public Safety System” represents the database structure and relationships between major system entities. The diagram includes entities such as USER, REFRESH TOKEN, ALERT, LOST PERSON REPORT, and CAMERA. The USER entity manages authentication details and role-based access for administrators and volunteers, while REFRESH TOKEN maintains active user sessions. The ALERT entity stores overcrowding notifications generated by the system, and LOST PERSON REPORT manages missing person information submitted by volunteers. The CAMERA entity represents surveillance sources used for crowd monitoring. The ER diagram helps maintain structured database relationships, data consistency, and efficient information management within the system.

5.6 Class Diagram

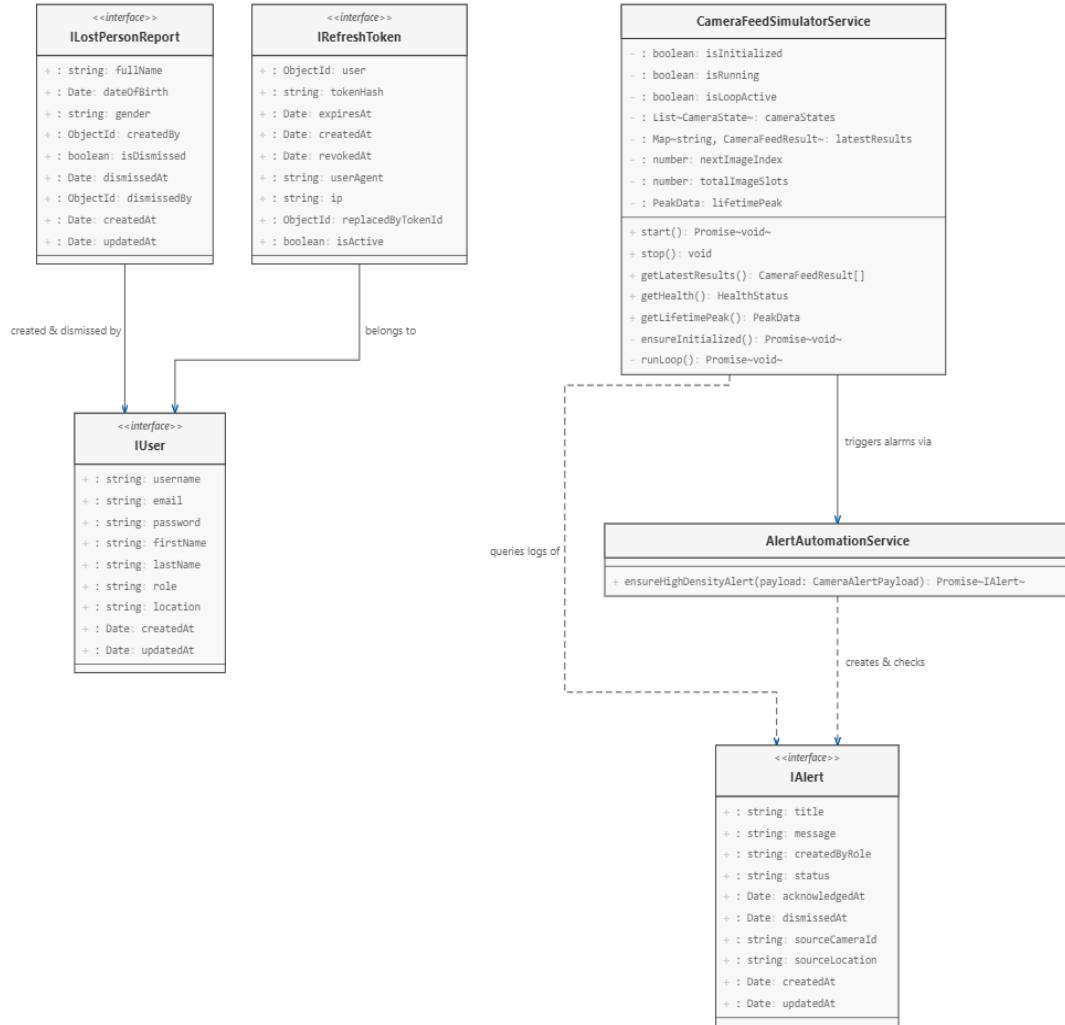


Figure 5.6: Class Diagram

The UML Class Diagram represents the static structure of the application by defining its major classes, attributes, methods, and relationships. The diagram includes core service classes such as `CameraFeedSimulatorService` and `AlertAutomationService` along with database model interfaces including `IUser`, `IRefreshToken`, `IAlert`, and `ILostPersonReport`. The `CameraFeedSimulatorService` processes crowd data and interacts with the `AlertAutomationService` for overcrowding detection and alert generation. User-related classes manage authentication, session handling, and role-based access control, while missing person and alert classes manage system records and safety operations.

5.7 Component Diagram

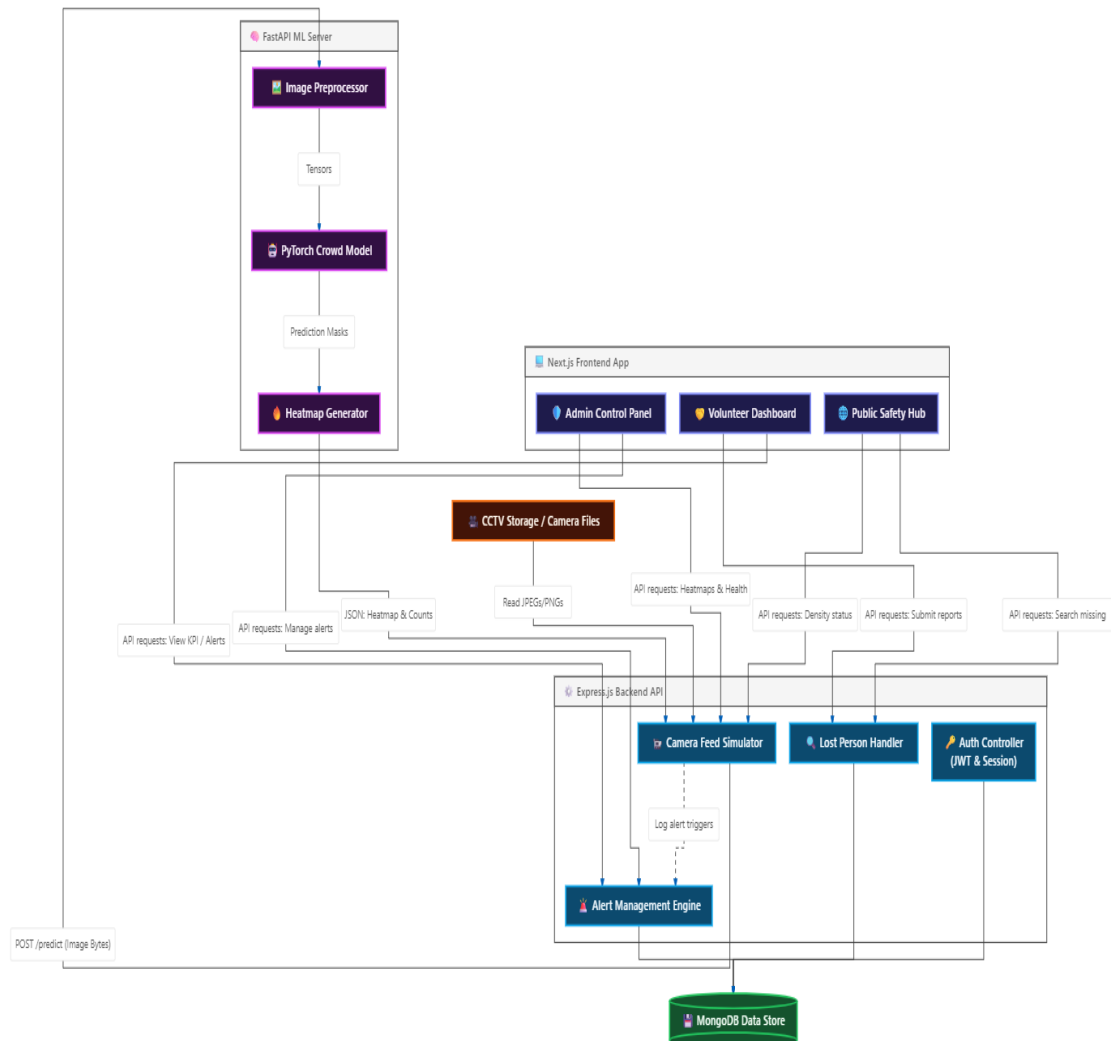


Figure 5.7: Component Diagram

The component diagram maps out the modular interaction between the Next.js Frontend Client, the Express.js Backend API, the FastAPI Machine Learning Server, and the MongoDB Persistence Store. The frontend client consists of three distinct, secure portals (Admin, Volunteer, and Public interfaces) that communicate with the backend's Auth, Alert, and Lost Person controllers via secure REST APIs. Simultaneously, the Camera Feed Simulator component reads raw files from local CCTV folders and forwards them via HTTP to the FastAPI server's Preprocessor and PyTorch Inference modules to calculate crowd metrics and generate thermal heatmaps. These calculated results are returned to the backend to update the database, update dashboards, and feed the automated Alert Engine, illustrating a highly decoupled, scalable, and service-oriented system architecture.

5.8 State Machine Diagram

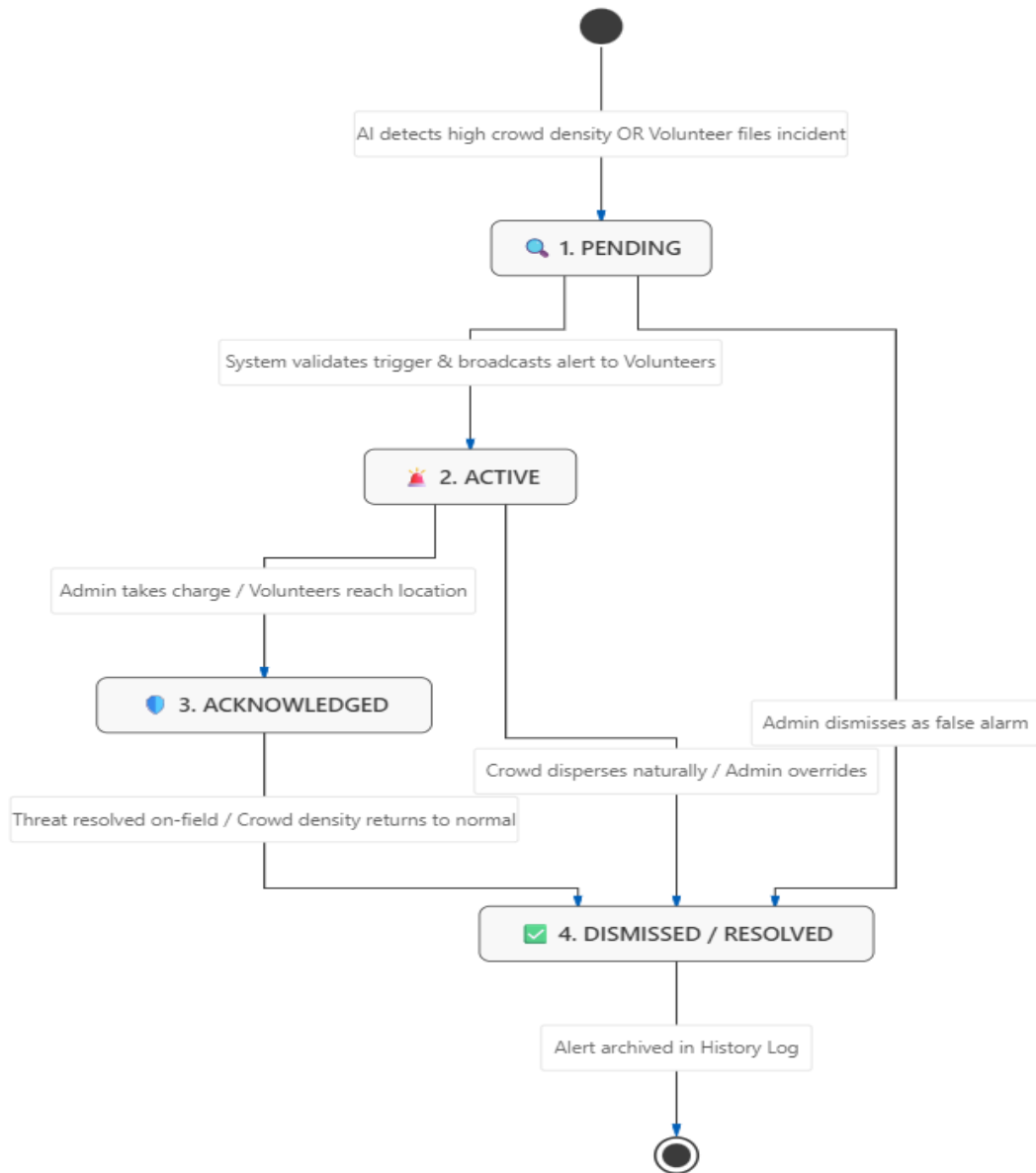


Figure 5.8: State Machine Diagram

The UML State Machine Diagram represents the lifecycle of a security alert generated during crowd monitoring operations. The alert transitions through different states such as PENDING, ACTIVE, ACKNOWLEDGED, and DISMISSED based on system events and administrator actions. An alert is initially created when overcrowding or abnormal crowd conditions are detected. After system validation, the alert becomes active and notifications are sent to administrators and volunteers. Once the issue is reviewed and handled by authorities, the alert is acknowledged and finally dismissed after the crowd situation is resolved. The diagram helps visualize the event-driven workflow and alert management process within the system.

5.9 Sequence Diagram

The Sequence Diagram illustrates the chronological interaction between different system components and actors involved in the AI-Powered Crowd Management and Public Safety System. It represents how requests, responses, and notifications are exchanged in real time to ensure efficient crowd monitoring and incident management. The process begins when the System Admin or monitoring module sends crowd image data to the Express Backend, which forwards the request to the FastAPI Machine Learning Service for analysis.

The ML service preprocesses the received image, performs crowd density prediction using the trained PyTorch model, and generates a density heatmap overlay for visualization. After inference, the predicted crowd count and heatmap results are returned to the backend server. The backend then evaluates the crowd density level and determines whether the detected situation exceeds predefined safety thresholds.

If the crowd density is identified as HIGH, the system automatically triggers emergency alert mechanisms. Alert records are created and stored in the database for future tracking and auditing purposes. Simultaneously, real-time notifications are pushed to the volunteer dashboard and administrative monitoring interface. This enables rapid coordination between authorities and field volunteers during emergency situations.

Volunteers can review the alert information and submit manual incident assessment details based on ground-level observations. The System Admin can acknowledge alerts, update incident statuses, and monitor the latest crowd conditions through the live dashboard. The database continuously stores crowd metrics, heatmap images, alert history, and volunteer responses to maintain proper records and support future analysis.

Overall, the Sequence Diagram highlights the real-time communication, automated decision-making, and coordinated response workflow of the proposed system, ensuring faster incident handling, improved situational awareness, and enhanced public safety management.

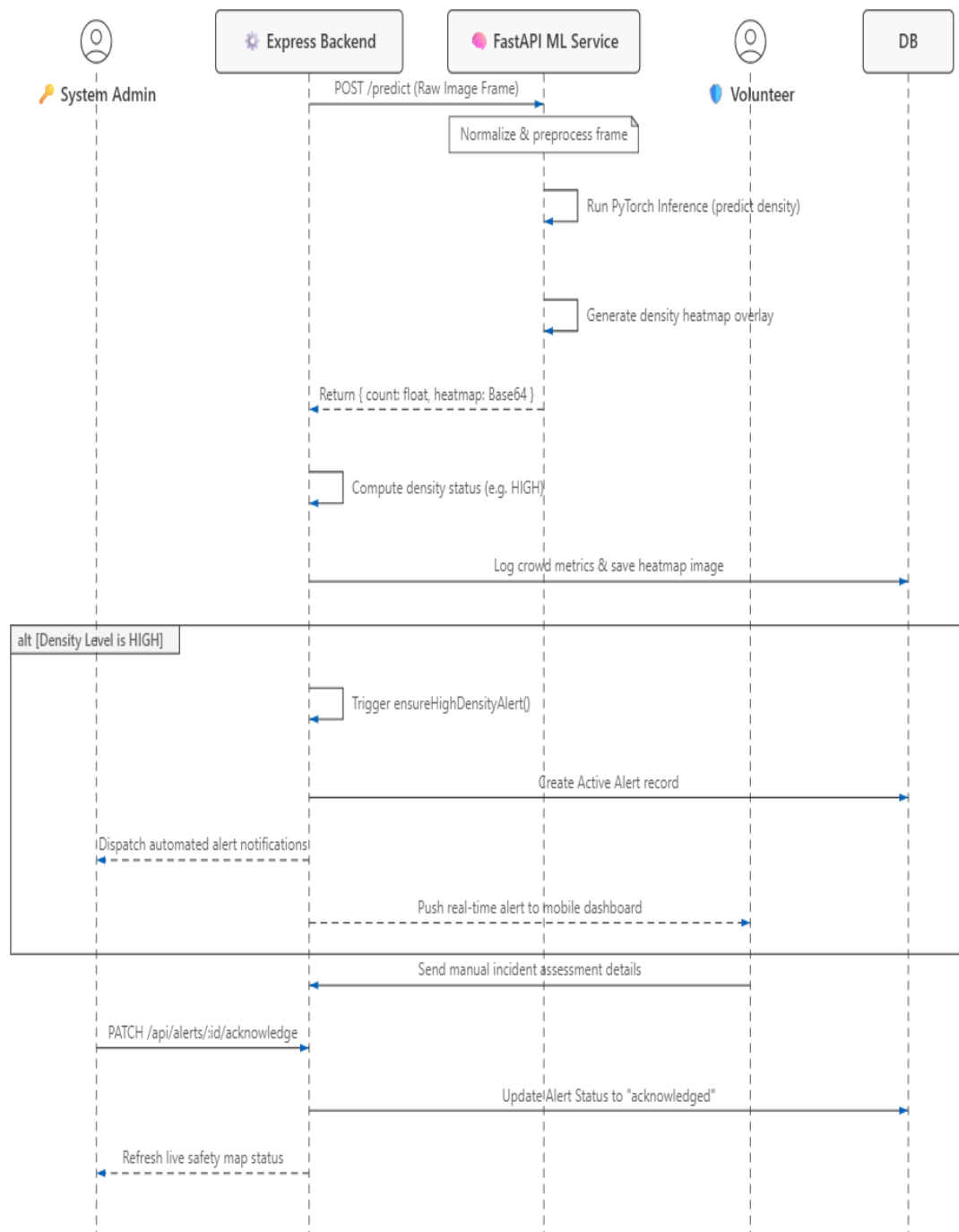


Figure 5.9: Sequence Diagram

5.10 Deployment Diagram

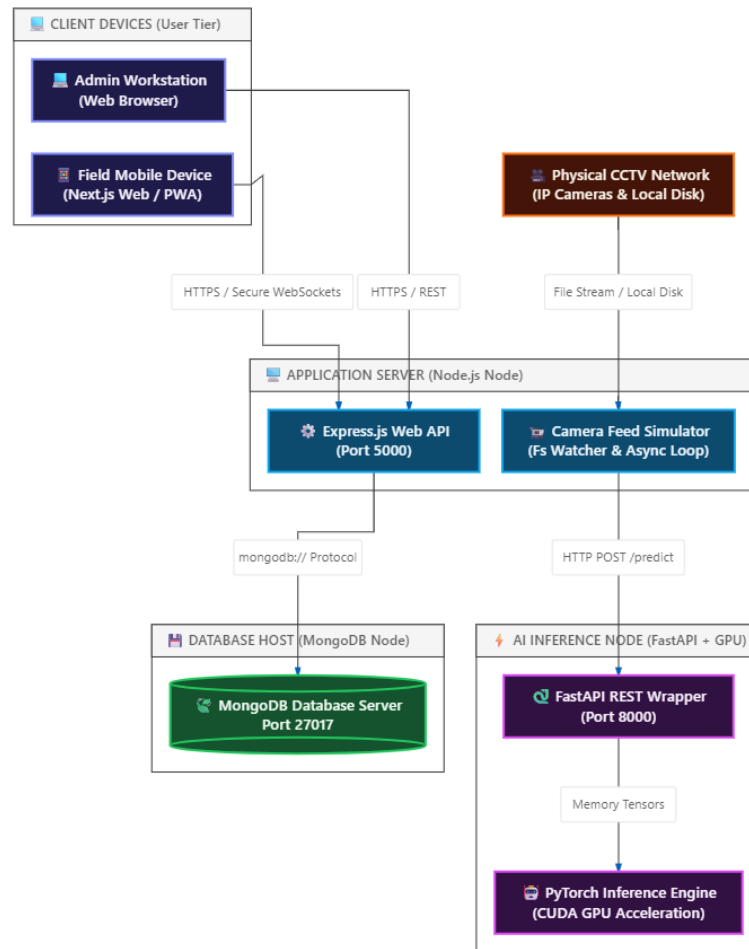


Figure 5.10: Deployment Diagram

The Deployment Diagram of the AI-Powered Crowd Management and Public Safety System represents the physical architecture of the system, showing how software components are deployed across various hardware nodes. It includes user devices (such as admin consoles, volunteer mobile apps, and public dashboards), the web server, AI processing server, and database server, all connected through the cloud network. External APIs like weather APIs and map APIs are also integrated to provide real-time data. This diagram helps visualize how different system components communicate and operate in a distributed environment to ensure scalability, reliability, and real-time performance.

Chapter 6

SYSTEM ARCHITECTURE

6.1 System Architecture

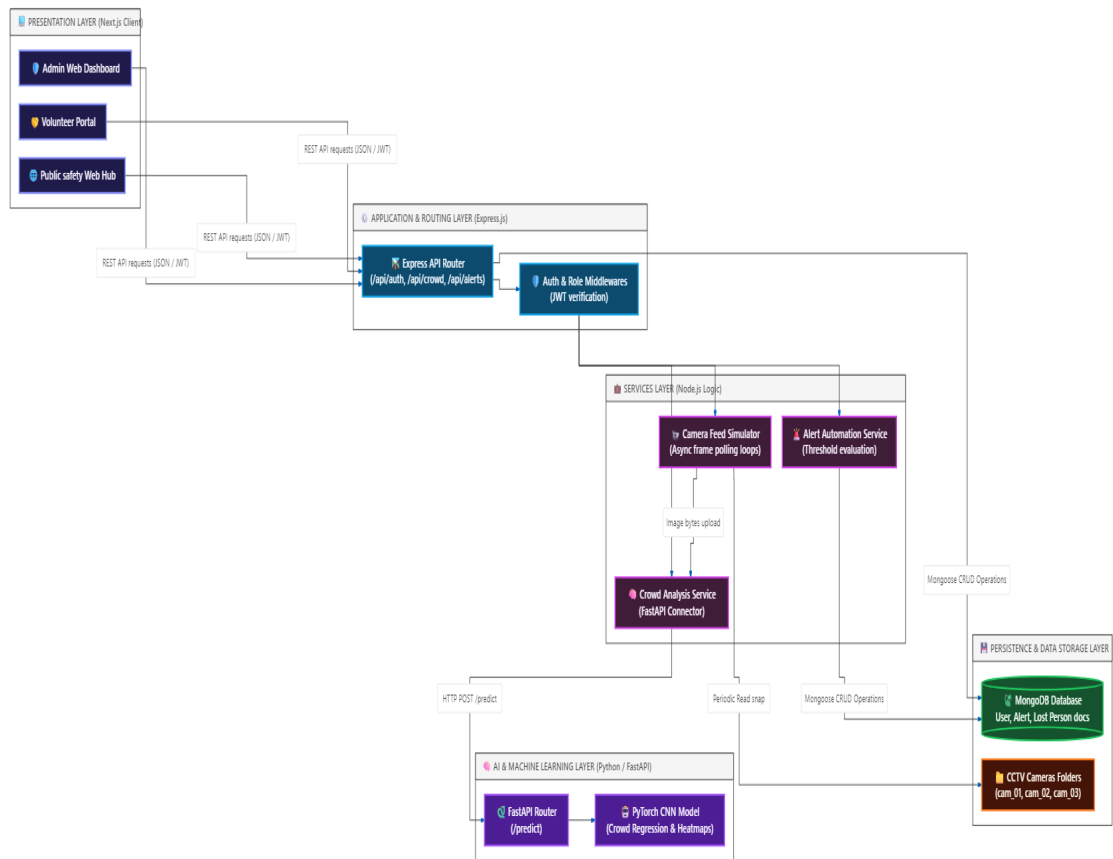


Figure 6.1: System Architecture Diagram

6.1.1 Data Acquisition and Preprocessing Layer

The Data Acquisition and Preprocessing Layer is responsible for collecting crowd image datasets and preparing the data for AI-based crowd density estimation. This layer ensures that the input data is properly formatted, standardized, and transformed before being passed to the deep learning model for training and inference.

The system uses the crowd dataset containing highly dense crowd images along with annotated ground truth points for density estimation. The acquired images are processed through multiple preprocessing operations to improve model performance and maintain consistency across the dataset.

Responsibilities:

- Collection and loading of crowd image datasets and annotation files.
- Validation of image format, annotation consistency, and dataset structure.
- Image resizing into fixed dimensions suitable for CNN-based processing.
- Generation of density maps using annotated crowd points.
- Image normalization and tensor transformation for AI model compatibility.
- Noise reduction and preprocessing of low-quality images.

Key Components:

- **Dataset Loader:** Loads crowd images and corresponding annotation files from the dataset.
- **Image Validator:** Verifies image integrity, resolution, and annotation consistency.
- **Image Resizer:** Converts images into standardized dimensions required by the AI model.
- **Density Map Generator:** Generates crowd density maps using Gaussian filtering techniques.
- **Normalization Engine:** Applies pixel normalization and tensor conversion operations.
- **Preprocessing Manager:** Controls the complete preprocessing workflow before data is passed to the CNN model.

Technologies Used:

- **OpenCV:** Image loading, resizing, and preprocessing operations.
- **NumPy:** Numerical processing and tensor manipulation.
- **SciPy:** Gaussian filtering and density map generation.
- **PyTorch:** Dataset handling and tensor transformation.

6.1.2 Feature Extraction and Transformation Layer

The Feature Extraction and Transformation Layer is responsible for extracting meaningful spatial information from preprocessed crowd images and transforming the image data into feature representations suitable for deep learning-based crowd density estimation. This layer improves the capability of the AI model to identify crowd regions, density variations, and overcrowded zones effectively.

The extracted image features are converted into structured feature maps that can be processed efficiently by the CNN-based CSRNet architecture. The layer also performs normalization and transformation operations to improve model stability and prediction accuracy.

Responsibilities:

- Extraction of spatial and visual features from crowd images.
- Transformation of image tensors into feature maps suitable for CNN processing.
- Normalization and scaling of extracted feature values.
- Enhancement of crowd-related spatial patterns for density estimation.
- Reduction of redundant or irrelevant feature information for optimized processing.

Key Components:

- **Feature Extractor:** Extracts important visual and spatial features from crowd images.
- **Transformation Engine:** Converts image tensors into structured feature representations.
- **Feature Scaler:** Performs normalization and scaling operations on extracted features.
- **Spatial Pattern Analyzer:** Identifies crowd-related spatial distributions and density variations.
- **Dimensionality Optimizer:** Reduces unnecessary feature complexity for efficient AI processing.

Technologies Used:

- **OpenCV:** Spatial image processing and feature extraction operations.
- **NumPy:** Numerical computations and feature tensor manipulation.
- **PyTorch:** Tensor transformation and CNN feature processing.
- **scikit-learn:** Feature normalization and preprocessing techniques.

6.1.3 AI-Based Crowd Density Estimation Layer

The AI-Based Crowd Density Estimation Layer is the core processing layer of the proposed system. This layer uses the CSRNet deep learning architecture to analyze crowd images and estimate crowd density using convolutional neural network techniques. The model generates density maps that represent the distribution and concentration of people present in the crowd image.

The CSRNet model uses a VGG16-based frontend network for extracting deep visual features and a dilated convolution backend for preserving spatial information during density estimation. The generated density maps are further used to calculate crowd count and identify overcrowded regions.

Responsibilities:

- Extraction of deep crowd features using CNN architecture.
- Generation of density maps for crowd density estimation.
- Calculation of estimated crowd count from predicted density maps.
- Identification of high-density and overcrowded areas based on predefined thresholds.
- Generation of heatmaps for real-time crowd visualization.

Key Components:

- **CSRNet Model:** Performs crowd density estimation using deep learning techniques.
- **VGG16 Frontend Network:** Extracts high-level visual features from crowd images.
- **Dilated Convolution Backend:** Preserves spatial resolution while generating density maps.
- **Density Prediction Engine:** Computes density maps and estimated crowd count.
- **Overcrowding Detection Module:** Identifies high-density crowd regions using threshold analysis.
- **Heatmap Generator:** Produces visual heatmaps for crowd density representation.

Technologies Used:

- **PyTorch:** Deep learning model implementation and training.
- **CSRNet:** Crowd density estimation architecture.
- **CNN:** Deep feature extraction and density prediction.
- **CUDA:** GPU acceleration for model training and inference.
- **OpenCV:** Heatmap generation and image visualization.

6.1.4 Model Training Layer

The Model Training Layer is responsible for training the CSRNet deep learning model using preprocessed crowd image datasets and generated density maps. This layer enables the system to learn crowd distribution patterns, density variations, and spatial characteristics required for accurate crowd density estimation.

During training, the model compares predicted density maps with ground truth density maps and minimizes prediction error using optimization algorithms and loss functions. The training process improves the ability of the model to estimate crowd count and identify overcrowded regions effectively.

Responsibilities:

- Training the CSRNet model using crowd image datasets and density maps.
- Performing forward propagation and density prediction.
- Calculating prediction loss between generated and ground truth density maps.
- Optimizing model parameters using backpropagation techniques.
- Saving trained model checkpoints for future inference and evaluation.
- Improving model accuracy through iterative training and optimization.

Key Components:

- **Training Engine:** Controls the complete model training workflow.
- **Loss Calculation Module:** Computes prediction error using Mean Squared Error (MSE) loss.
- **Optimizer Module:** Updates model weights using optimization algorithms.
- **Checkpoint Manager:** Stores trained model parameters and training progress.
- **Batch Processing Unit:** Handles batch-wise image and density map processing.
- **Gradient Scaling Module:** Improves training stability during mixed precision computation.

Technologies Used:

- **PyTorch:** Deep learning model training and tensor computation.
- **CUDA:** GPU-based accelerated model training.
- **Adam Optimizer:** Optimization algorithm for model parameter updates.
- **MSE Loss Function:** Loss calculation for density estimation accuracy.
- **Automatic Mixed Precision (AMP):** Optimized GPU memory utilization and faster computation.

6.1.5 Inference and Model Evaluation Layer

The Inference and Model Evaluation Layer is responsible for generating crowd density predictions using the trained CSRNet model and evaluating the performance of the model using various accuracy metrics. This layer performs real-time crowd count estimation, density map generation, heatmap visualization, and comparison between predicted and actual crowd counts.

During inference, the trained model processes input crowd images and generates density maps representing crowd distribution. The predicted density values are summed to estimate the total crowd count. The generated results are then evaluated using statistical performance metrics to measure model accuracy and reliability.

Responsibilities:

- Loading the trained CSRNet model for prediction.
- Performing crowd density estimation on input crowd images.
- Generating predicted density maps and crowd count values.
- Creating visual heatmaps for crowd density representation.
- Comparing predicted crowd counts with actual ground truth counts.
- Evaluating model performance using statistical accuracy metrics.

Key Components:

- **Inference Engine:** Performs real-time crowd density prediction using the trained model.
- **Density Prediction Module:** Generates density maps from input crowd images.
- **Crowd Count Estimator:** Calculates total crowd count from predicted density maps.
- **Heatmap Visualization Module:** Generates visual crowd heatmaps for density analysis.
- **Performance Evaluation Module:** Measures prediction accuracy using evaluation metrics.
- **Ground Truth Comparator:** Compares actual and predicted crowd counts.

Evaluation Metrics:

- **Mean Absolute Error (MAE):** Measures the average absolute difference between predicted and actual crowd counts.
- **Root Mean Squared Error (RMSE):** Measures prediction error magnitude and model consistency.

- **Prediction Error Analysis:** Calculates image-wise prediction deviation for model evaluation.
- **Actual vs Predicted Comparison:** Compares ground truth crowd count with estimated crowd count.

Technologies Used:

- **PyTorch:** Model inference and tensor computation.
- **OpenCV:** Heatmap generation and image visualization.
- **Matplotlib:** Visualization of prediction results and crowd density outputs.
- **NumPy:** Numerical computations and density map processing.

6.1.6 Backend Integration and API Communication Layer

The Backend Integration and API Communication Layer is responsible for establishing communication between the AI-based crowd density estimation model and the web application. This layer manages API requests, transfers prediction results from the Flask server to the frontend dashboard, and handles system-level processing operations.

The trained CSRNet model is deployed using the Flask framework, where prediction requests are processed and crowd density results are returned to the frontend application. The backend layer also manages alert generation, database communication, and integration with external APIs.

Responsibilities:

- Integration of the trained AI model with backend services.
- Handling API requests and responses between frontend and AI modules.
- Processing crowd density prediction results.
- Managing overcrowding alerts and system notifications.
- Communicating with database services for storing crowd-related information.
- Integration of weather and map-based external APIs.

Key Components:

- **Flask API Server:** Hosts the trained CSRNet model and processes prediction requests.
- **API Request Handler:** Manages communication between frontend and backend services.
- **Prediction Response Manager:** Transfers crowd density results to the dashboard.

- **Alert Processing Module:** Generates overcrowding alerts based on density thresholds.
- **External API Connector:** Handles communication with weather and map APIs.

Technologies Used:

- **Flask:** AI model deployment and API communication.
- **REST APIs:** Communication between frontend and backend modules.
- **Python:** Backend processing and AI integration.
- **JSON:** Data transfer and API response formatting.

6.1.7 Database Management Layer

The Database Management Layer is responsible for securely storing and managing crowd-related information generated by the system. This layer maintains records associated with crowd density analysis, alerts, user information, missing person reports, and environmental updates. The database layer ensures efficient storage, retrieval, and synchronization of information required by the frontend dashboard and backend services.

Responsibilities:

- Storage of crowd density logs and prediction results.
- Management of automatic alerts and dismissal.
- Storage of user authentication and role-based access information.
- Management of missing person reports and related details.
- Storage of environmental and weather-related information.

Key Components:

- **Crowd Data Repository:** Stores crowd density and prediction records.
- **Alert Database Module:** Maintains overcrowding alert history.
- **User Management Module:** Stores authentication and user role information.
- **Missing Person Repository:** Maintains lost and found person records.
- **Database Access Manager:** Handles database communication and query execution.

Technologies Used:

- **MongoDB:** NoSQL database management and cloud storage.
- **Mongoose / Database Drivers:** Database communication and schema management.

6.1.8 Frontend Visualization Layer

The Frontend Visualization Layer provides a web-based interface for monitoring crowd density information, heatmap visualization, alerts, and environmental updates. This layer enables administrators, volunteers, and users to interact with the system through a responsive and real-time dashboard.

The frontend dashboard continuously fetches updated crowd information from backend APIs using polling mechanisms to maintain near real-time visualization of crowd conditions and system alerts.

Responsibilities:

- Displaying crowd density heatmaps and visual analytics.
- Showing overcrowding alerts and notification updates.
- Displaying weather and environmental information.
- Providing role-based dashboard access for Admin, Volunteer, and User.
- Continuously fetching updated crowd information using polling techniques.
- Visualizing AI prediction results in an interactive dashboard interface.

Key Components:

- **Dashboard Interface:** Displays real-time crowd monitoring information.
- **Heatmap Visualization Module:** Visualizes crowd density distribution using color-coded regions.
- **Alert Display Module:** Shows system-generated alerts and notifications.
- **Polling Manager:** Periodically fetches updated crowd data from backend APIs.
- **User Authentication Module:** Handles login and role-based access management.

Technologies Used:

- **Next.js:** Frontend framework for dashboard development.
- **React.js:** Component-based user interface development.
- **TypeScript:** Type-safe frontend application development.
- **Tailwind CSS / ShadCN UI:** User interface styling and component design.
- **OpenStreetMap API:** Map-based visualization support.
- **OpenWeatherMap API:** Environmental and weather information integration.

Chapter 7

WORKING METHODOLOGY

7.1 Methodological Strategy

The Trinetra – AI-Powered Crowd Management and Public Safety System development follows a structured, data-driven methodology emphasizing real-time processing, public safety, and rigorous evaluation.

7.1.1 Data Acquisition and Preprocessing Layer

Approach: The Data Acquisition and Preprocessing Layer focuses on collecting crowd image datasets and preparing them for AI-based crowd density estimation. The system uses the UCF-QNRF dataset containing highly dense crowd images along with annotated crowd points. The acquired images are validated and transformed into standardized formats before being passed to the AI model. The preprocessing operations include image resizing, normalization, density map generation, and tensor conversion to ensure compatibility with the CNN-based CSRNet architecture.

Strategies: The system applies image standardization strategies by resizing all images into fixed dimensions for maintaining uniformity during model training and inference. Gaussian filtering techniques are used for generating density maps from annotated crowd points to improve spatial crowd representation. Noise reduction and normalization techniques are also applied to enhance image quality and improve model performance. OpenCV, NumPy, SciPy, and PyTorch are used for preprocessing, tensor manipulation, and density map generation operations.

7.1.2 Feature Extraction and Transformation Layer

Approach: The Feature Extraction and Transformation Layer is responsible for extracting meaningful spatial and visual information from preprocessed crowd images. The system transforms input images into feature representations suitable for deep learning-based crowd density estimation. This layer helps the AI model learn crowd distributions, density variations, and important spatial patterns required for accurate crowd analysis.

Strategies: The system uses convolutional neural network-based feature extraction

techniques to identify spatial crowd patterns from input images. Feature normalization and scaling methods are applied to improve model stability and prediction accuracy. Image tensors are transformed into structured feature maps that can be efficiently processed by the CSRNet architecture. Dimensional optimization techniques are also used to reduce unnecessary feature complexity and improve computational efficiency. OpenCV, NumPy, PyTorch, and scikit-learn are used for feature extraction, normalization, and tensor transformation processes.

7.1.3 AI-Based Crowd Density Estimation Layer

Approach: The AI-Based Crowd Density Estimation Layer performs crowd density analysis using the CSRNet deep learning architecture. The system processes transformed feature maps to generate density maps representing crowd distribution within the image. The predicted density values are summed to estimate the total crowd count and identify overcrowded regions.

Strategies: The system uses a VGG16-based frontend network for extracting high-level visual features from crowd images. Dilated convolution layers are used in the backend architecture to preserve spatial resolution while generating high-quality density maps. Threshold-based analysis is used for identifying high-density and overcrowded crowd regions. Heatmap generation strategies are also implemented for visual crowd density representation. PyTorch, CSRNet architecture, CNN models, CUDA acceleration, and OpenCV are used for density estimation, heatmap generation, and AI processing operations.

7.1.4 Model Training Layer

Approach: The Model Training Layer is responsible for training the CSRNet model using preprocessed crowd images and generated density maps. The model learns spatial crowd distributions and density estimation patterns through supervised deep learning techniques. During training, predicted density maps are compared with ground truth density maps to improve model accuracy.

Strategies: The system uses forward propagation and backpropagation strategies for optimizing model parameters during training. Mean Squared Error (MSE) is used as the loss function to minimize prediction error between predicted and actual density maps. The Adam optimizer is used for updating model weights iteratively. Automatic Mixed Precision (AMP) and CUDA-based GPU acceleration techniques are applied to improve computational efficiency and training speed. Model checkpoint strategies are also implemented for saving trained model parameters and maintaining training progress.

7.1.5 Inference and Model Evaluation Layer

Approach: The Inference and Model Evaluation Layer performs crowd density prediction using the trained CSRNet model and evaluates model performance using multiple statistical metrics. The system processes crowd images to generate density maps, estimate crowd count, and visualize crowd density distributions through heatmaps.

Strategies: The system uses density map summation techniques for calculating estimated crowd count from predicted density outputs. Heatmap visualization strategies are implemented using color mapping techniques for easier crowd density interpretation. Model performance is evaluated by comparing predicted crowd counts with actual ground truth counts. Statistical evaluation metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) are used to measure prediction accuracy and model consistency. PyTorch, OpenCV, Matplotlib, and NumPy are used for inference, visualization, and performance evaluation processes.

7.1.6 Backend Integration and API Communication Layer

Approach: The Backend Integration and API Communication Layer manages communication between the AI model and the web application. The trained CSRNet model is deployed using Flask APIs, allowing prediction requests and crowd density information to be exchanged between the backend and frontend dashboard.

Strategies: The system uses REST API-based communication strategies for transferring crowd density predictions, heatmap information, and alert data between modules. Flask is used for hosting the AI model and processing prediction requests. JSON-based data transfer strategies are implemented for efficient frontend-backend communication. The backend also handles alert processing, database communication, and external API integration. Python, Flask, REST APIs, and JSON are used for backend processing and API communication.

7.1.7 Database Management Layer

Approach: The Database Management Layer is responsible for securely storing and managing crowd-related information generated by the system. The database maintains records associated with crowd density predictions, alerts, user details, environmental updates, and missing person information.

Strategies: The system uses centralized NoSQL database management strategies for efficient storage and retrieval of crowd monitoring information. MongoDB is used to maintain scalable and flexible document-based storage. Database synchronization strategies are implemented to support continuous updating of crowd logs, alerts, and user records. Efficient query management techniques are also used for rapid retrieval of crowd-related information required by the frontend dashboard and backend services.

7.1.8 Frontend Visualization Layer

Approach: The Frontend Visualization Layer provides a web-based dashboard for displaying crowd density information, heatmaps, alerts, and environmental updates. The dashboard enables administrators, volunteers, and users to monitor crowd conditions and visualize AI-generated crowd analytics interactively.

Strategies: The system uses polling-based communication strategies for continuously fetching updated crowd information from backend APIs. Heatmap visualization strategies are implemented using color-coded crowd density representations for

easier monitoring of overcrowded regions. Responsive dashboard design techniques are used to support both desktop and mobile interfaces. OpenStreetMap API is integrated for map visualization, while OpenWeatherMap API provides environmental and weather-related updates. Next.js, React.js, TypeScript, and frontend visualization libraries are used for dashboard development and user interaction.

7.2 Project Modules

7.2.1 Interface

This section contains the Admin Command Center Interface showing the Key Performance Index Cards and Crowd Monitoring module that shows density heatmap and crowd count.

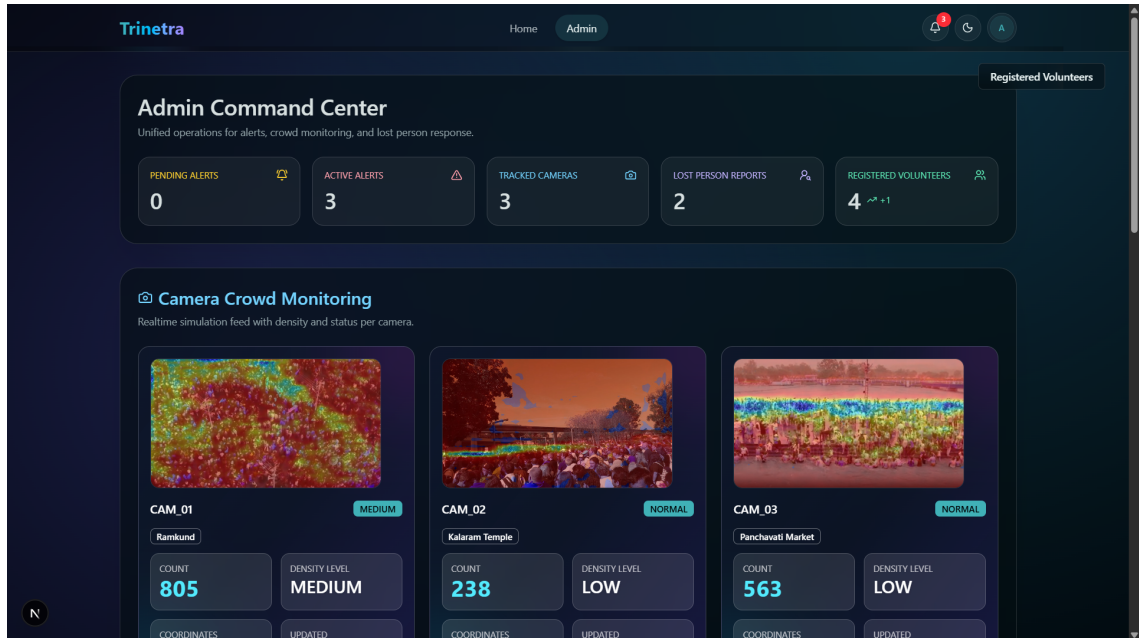
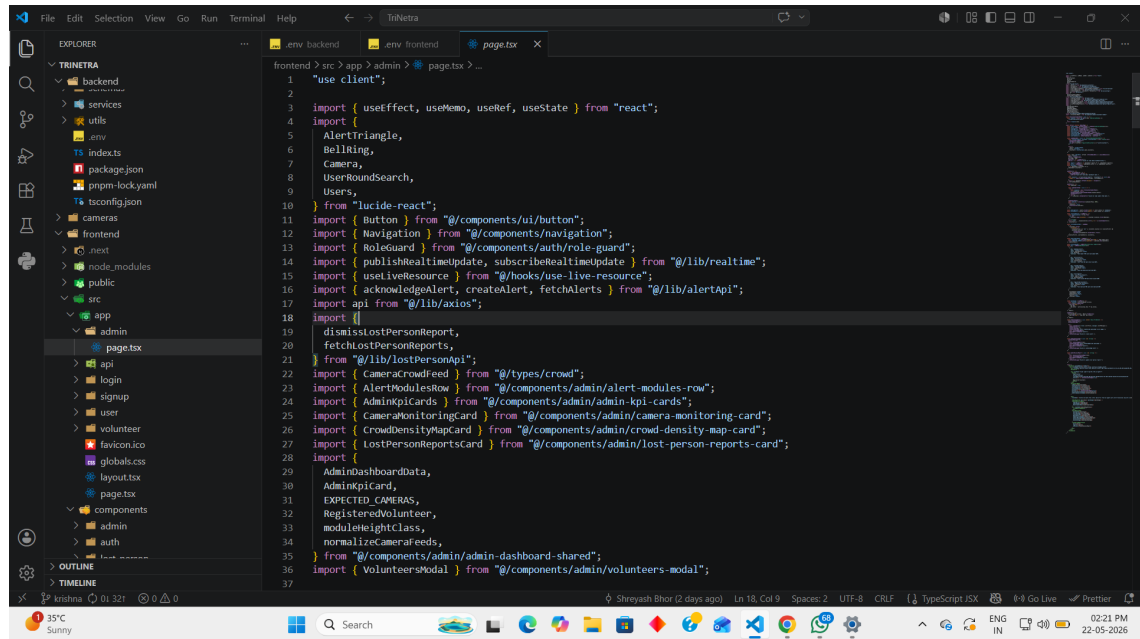


Figure 7.1: User Interface

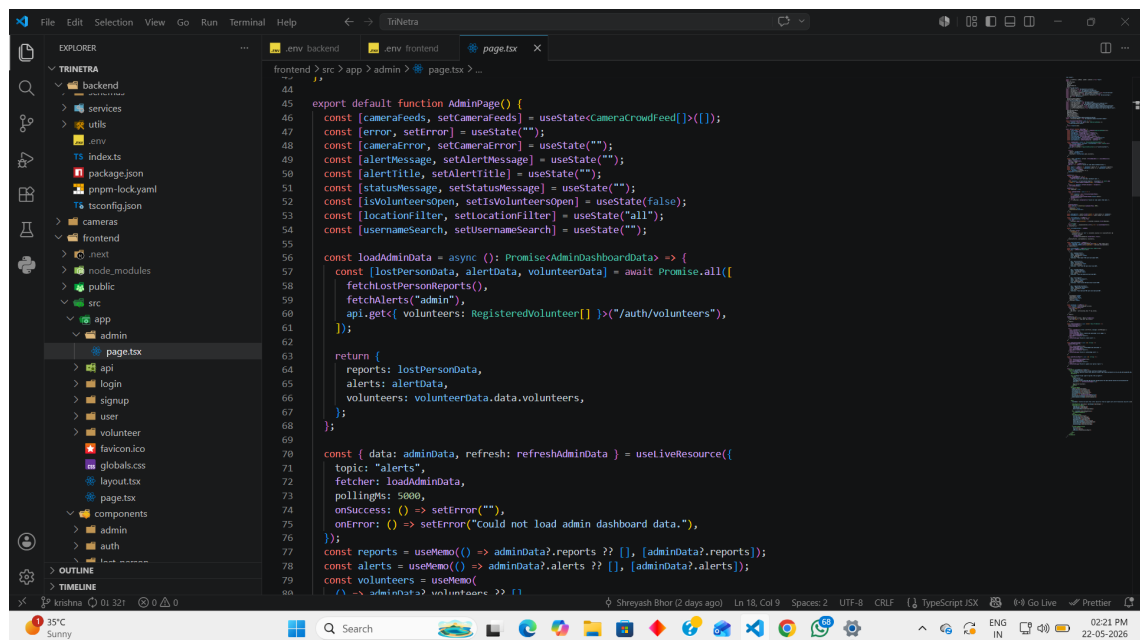
7.2.2 Code Snippets

In this section, we provide key code snippets used in the development of the project.



```
1 "use client";
2
3 import { useEffect, useMemo, useRef, useState } from "react";
4 import {
5   AlertTriangle,
6   BellRing,
7   Camera,
8   UserRoundSearch,
9   Users,
10 } from "lucide-react";
11 import { Button } from "@components/ui/button";
12 import { Navigation } from "@components/navigation";
13 import { RoleGuard } from "@components/auth/role-guard";
14 import { publishRealtimeUpdate, subscribeRealtimeUpdate } from "@lib/realtime";
15 import { useLiveResource } from "@hooks/use-live-resource";
16 import { acknowledgeAlert, createAlert, fetchAlerts } from "@lib/alertApi";
17 import api from "@lib/axios";
18 import {
19   dismissLostPersonReport,
20   fetchLostPersonReports,
21 } from "@lib/lostPersonApi";
22 import { CameraCrowdFeed } from "@types/crowd";
23 import { AlertModuleRow } from "@components/admin/alert-modules-row";
24 import { AdminKpiCards } from "@components/admin/admin-kpi-cards";
25 import { CameraMonitoringCard } from "@components/admin/camera-monitoring-card";
26 import { CrowdDensityMapCard } from "@components/admin/crowd-density-map-card";
27 import { LostPersonReportsCard } from "@components/admin/lost-person-reports-card";
28 import {
29   AdminDashboardData,
30   AdminKpiCard,
31   EXPECTED_CAMERAS,
32   RegisteredVolunteer,
33   moduleHeightClass,
34   normalizeCameraFeeds,
35 } from "@components/admin/admin-dashboard-shared";
36 import { VolunteersModal } from "@components/admin/volunteers-modal";
37
```

Figure 7.2: Implementation Code



```
44
45 export default function AdminPage() {
46   const [cameraFeeds, setCameraFeeds] = useState<CameraCrowdFeed[]>([]);
47   const [error, setError] = useState("");
48   const [cameraError, setCameraError] = useState("");
49   const [alertMessage, setAlertMessage] = useState("");
50   const [alertTitle, setAlertTitle] = useState("");
51   const [statusMessage, setStatusMessage] = useState("");
52   const [isVolunteersOpen, setIsVolunteersOpen] = useState(false);
53   const [locationFilter, setLocationFilter] = useState("");
54   const [usernameSearch, setUsernameSearch] = useState("");
55
56   const loadAdminData = async (): Promise<AdminDashboardData> => {
57     const [lostPersonData, alertData, volunteerData] = await Promise.all([
58       fetchLostPersonReports(),
59       fetchAlerts("admin"),
60       api.get<{ volunteers: RegisteredVolunteer[] }>("/auth/volunteers"),
61     ]);
62
63     return {
64       reports: lostPersonData,
65       alerts: alertData,
66       volunteers: volunteerData.data.volunteers,
67     };
68   };
69
70   const { data: adminData, refresh: refreshAdminData } = useLiveResource({
71     topic: "alerts",
72     fetcher: loadAdminData,
73     pollingMs: 5000,
74     onSuccess: () => setError(""),
75     onError: () => setError("Could not load admin dashboard data."),
76   });
77
78   const reports = useMemo(() => adminData?.reports ?? [], [adminData?.reports]);
79   const alerts = useMemo(() => adminData?.alerts ?? [], [adminData?.alerts]);
80   const volunteers = useMemo(
81     () => adminData?.volunteers ?? [],
82     [adminData?.volunteers]
83   );
84
```

Figure 7.3: Implementation Code

Chapter 8

TESTING AND RESULTS

8.1 Testing

Testing is an important phase of the implemented system “TRINETRA – AI Powered Crowd Management and Public Safety System” used to verify the functionality, accuracy, reliability, and performance of the complete application. The testing process was conducted on the AI-based crowd density estimation model, backend APIs, database operations, and frontend dashboard to ensure proper system integration and stable execution.

The AI model was tested using crowd image datasets containing highly dense crowd scenes. The testing process verified the ability of the CSRNet model to generate accurate density maps and estimate crowd count effectively. Different crowd images were processed through the inference pipeline to evaluate crowd density prediction accuracy and heatmap generation performance.

The backend services and Flask APIs were tested to verify communication between the AI model and frontend dashboard. API responses, prediction request handling, alert generation, and data transfer mechanisms were validated to ensure smooth system operation. Database testing was also performed to verify secure storage and retrieval of crowd logs, alert records, and user information using MongoDB. The frontend dashboard developed using Next.js, React.js, and TypeScript was tested for responsive visualization, heatmap rendering, polling-based data updates, and alert display functionalities. User interface testing was conducted to ensure proper interaction between dashboard components and backend services.

Performance testing was conducted to evaluate the efficiency of crowd density estimation and overall system responsiveness. Security testing was also performed to verify authentication mechanisms, protected API communication, and secure data handling processes.

8.2 Testing Strategies

8.2.1 Functional Testing

Functional testing was performed to verify whether all modules of the system were operating according to the specified requirements. The testing process included crowd density estimation, heatmap generation, overcrowding detection, alert display, polling-based dashboard updates, API communication, and database operations.

8.2.2 Integration Testing

Integration testing was conducted to ensure proper communication between the AI model, Flask backend, MongoDB database, and frontend dashboard. The testing verified the successful transfer of crowd density predictions, alerts, and visualization data between all system layers.

8.2.3 Performance Testing

Performance testing was carried out to evaluate model inference speed, frontend responsiveness, API response time, and overall system efficiency. GPU acceleration and optimized preprocessing techniques were used to improve computational performance during AI inference.

8.2.4 Security Testing

Security testing was performed to validate authentication mechanisms, secure API communication, and protected database access. The testing ensured that unauthorized users could not access sensitive crowd-related information or system controls.

8.3 Results

The implemented system successfully performed crowd density estimation and overcrowding identification using the CSRNet deep learning architecture. The generated density maps accurately represented crowd distribution within highly dense crowd images, while the heatmap visualization modules provided clear visual interpretation of crowd density levels.

The AI model successfully estimated crowd count by generating density maps from crowd images processed through the CNN-based architecture. The generated heatmaps effectively highlighted high-density and overcrowded regions using color-based visualization techniques.

The frontend dashboard successfully displayed crowd heatmaps, overcrowding alerts, crowd statistics, environmental updates, and missing person information through an interactive interface. Polling-based communication mechanisms continuously retrieved updated crowd information from backend APIs to maintain near real-time

dashboard synchronization.

The Flask backend successfully handled prediction requests and transferred crowd density information between the AI model and frontend application. MongoDB efficiently managed crowd logs, alerts, user records, and other system-related information.

The overall implementation demonstrated stable integration between AI processing, backend communication, database management, and frontend visualization modules. The system achieved reliable crowd density estimation performance and successfully supported intelligent crowd monitoring and public safety management operations.

8.4 Performance Evaluation Metrics

The performance of the CSRNet crowd density estimation model was evaluated using statistical accuracy metrics to measure prediction reliability and crowd count estimation performance.

8.4.1 Mean Absolute Error (MAE):

Mean Absolute Error was used to measure the average absolute difference between the predicted crowd count and the actual ground truth crowd count. Lower MAE values indicate better prediction accuracy.

8.4.2 Mean Absolute Error (MAE):

Mean Absolute Error was used to measure the average absolute difference between the predicted crowd count and the actual ground truth crowd count. The proposed CSRNet-based crowd counting model achieved an MAE of 485.87 on the dataset, indicating the average prediction deviation across highly dense crowd scenes.

8.4.3 Root Mean Squared Error (RMSE):

Root Mean Squared Error was used to evaluate overall prediction consistency and model robustness by penalizing larger prediction errors more heavily. The developed model achieved an RMSE of 841.73, reflecting the challenges associated with extremely dense and perspective-distorted crowd images present in the dataset.

8.4.4 Prediction Accuracy within Error Thresholds:

Additional evaluation metrics were implemented to measure prediction accuracy under different acceptable error tolerances. The model achieved 30.84% prediction accuracy within $\pm 10\%$ error, 55.09% accuracy within $\pm 20\%$ error, and 73.95% accuracy within $\pm 30\%$ error. These metrics provide a more practical evaluation of regression-based crowd counting performance.

8.4.5 Mean Prediction Accuracy:

The overall mean prediction accuracy of the proposed model was measured using percentage-based prediction correctness across all test samples. The CSRNet model achieved a mean prediction accuracy of 79.43%, demonstrating reliable crowd estimation capability on highly dense crowd scenes.

8.4.6 Crowd Density Heatmap Visualization:

Heatmap visualization was implemented to analyze spatial crowd distribution and identify highly congested regions within surveillance images. The generated density heatmaps visually represent crowd concentration intensity and assist in understanding model inference behavior.

8.4.7 Actual vs Predicted Crowd Count Comparison:

The predicted crowd counts generated by the CSRNet model were compared against annotated ground truth counts from density maps to evaluate inference performance. Graphical comparison between actual and predicted counts was used to analyze estimation trends and prediction consistency across the test dataset.

8.4.8 Confusion Matrix:

A multi-class confusion matrix approach was implemented by categorizing crowd counts into Low, Medium, and High density classes. This classification-based evaluation was used to analyze the model's ability to distinguish varying crowd density levels within highly populated scenes.

Chapter 9

RESULTS

9.1 Output

9.1.1 Crowd Density Heatmap Result:

The generated crowd density heatmap visually represents the spatial distribution of people detected within the surveillance image. High-density regions are highlighted using warmer color intensities, enabling easier identification of overcrowded areas in real time. The CSRNet model predicted a crowd count of 655.71 compared to the actual ground truth count of 601, resulting in an error of 54.71. The visualization demonstrates that the proposed model is capable of accurately estimating crowd concentration while preserving important spatial crowd distribution patterns.

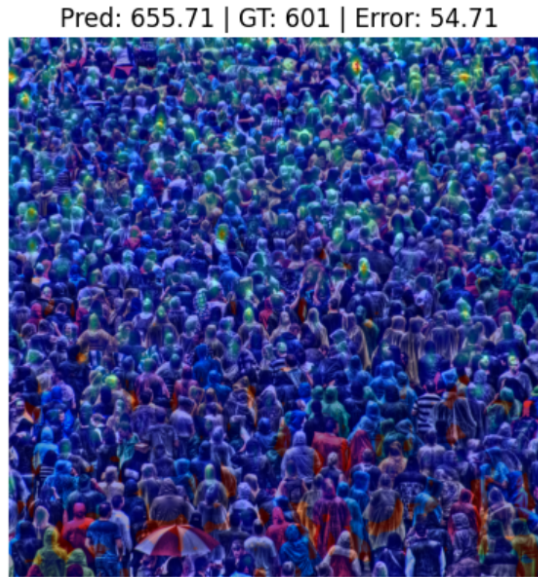


Figure 9.1: Density Heatmap

9.1.2 Confusion Matrix Result:

The confusion matrix was used to evaluate the crowd density classification capability of the proposed CSRNet model. The model correctly classified 249 high-density crowd images and 48 low-density crowd images, demonstrating effective recognition of dense crowd patterns. A small number of misclassifications were observed, including 11 false positives and 26 false negatives, indicating moderate confusion between density categories. Overall, the confusion matrix analysis confirms that the model performs reliably in distinguishing varying crowd density levels within the dataset.

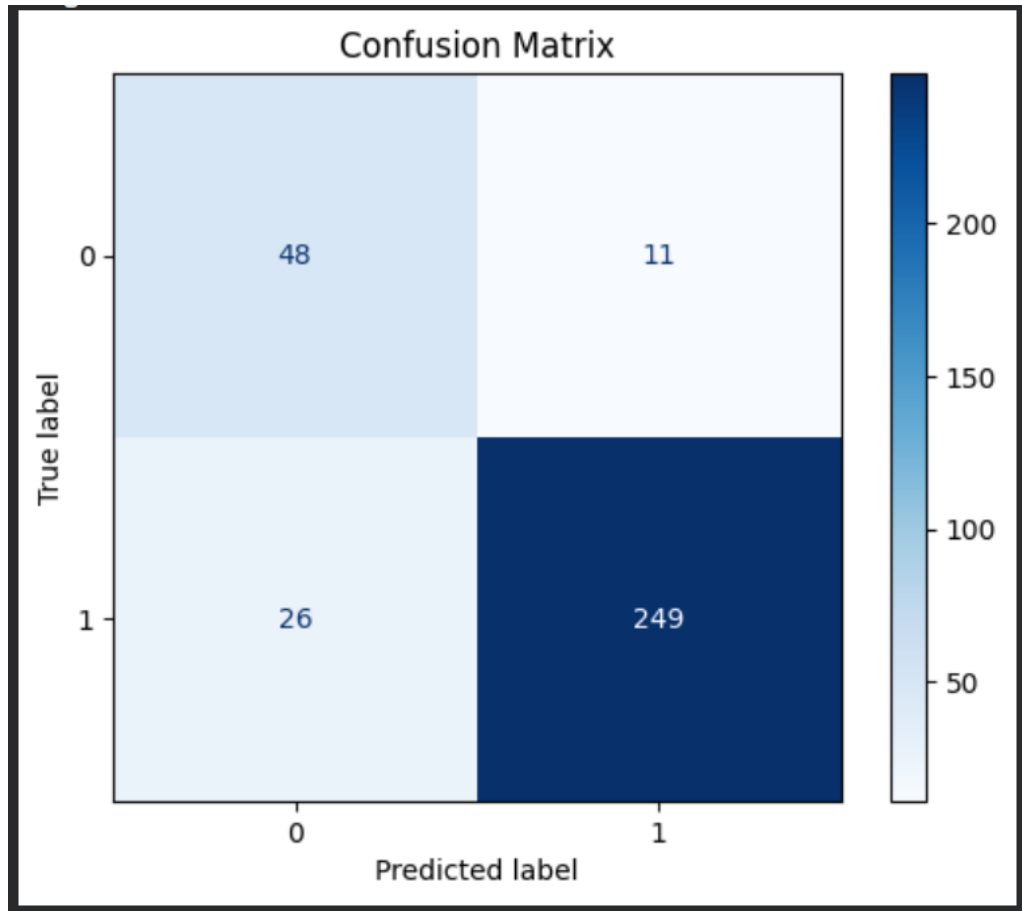


Figure 9.2: Confusion Matrix

9.1.3 Advanced Model Metrics Result:

Advanced evaluation metrics were implemented to analyze the prediction accuracy and robustness of the proposed crowd counting model. The model achieved 29.64% accuracy within $\pm 10\%$ error, 57.49% accuracy within $\pm 20\%$ error, and 77.25% accuracy within $\pm 30\%$ error tolerance. The overall mean prediction accuracy obtained was 79.65%, while the mean error percentage was measured as 20.49%, indicating stable performance across highly dense crowd scenes. Additionally, the best prediction achieved an error of only 0.13%, demonstrating the model's capability to generate highly accurate crowd estimates for several challenging test images.

```
-----  
ADVANCED MODEL METRICS  
-----  
Accuracy within  $\pm 10\%$  : 29.64%  
Accuracy within  $\pm 20\%$  : 57.49%  
Accuracy within  $\pm 30\%$  : 77.25%  
  
Mean Prediction Accuracy : 79.65%  
Mean Error Percentage    : 20.49%  
  
-----  
BEST PREDICTION  
-----  
Error % : 0.13%  
Image   : /content/drive/MyDrive/FinalModel/datasets/UCF-QNRF_ECCV18/Test/img_0108.jpg  
  
-----  
WORST PREDICTION  
-----  
Error % : 137.93%  
Image   : /content/drive/MyDrive/FinalModel/datasets/UCF-QNRF_ECCV18/Test/img_0081.jpg
```

Figure 9.3: Evaluation Metrics

Chapter 10

CONCLUSION & FUTURE SCOPE

10.1 Conclusion

The Trinetra – AI-Powered Crowd Management and Public Safety System successfully demonstrates how artificial intelligence and computer vision can transform crowd safety during large public gatherings. By integrating real-time CCTV-based video surveillance, CNN/CSRNet crowd density estimation, automated alert generation, and a web-based dashboard with heatmap visualization, the system minimizes the risk of overcrowding and enhances the efficiency of law enforcement, volunteers, and emergency response teams. The inclusion of missing person management, weather forecast and automatic alert system makes Trinetra a comprehensive and scalable public safety solution suitable for events ranging from local gatherings to large-scale events like the Kumbh Mela.

10.2 Future Scope

The Trinetra system has significant potential for future enhancements and large-scale real-world implementation. In future versions, the system can be integrated with advanced drone-based surveillance systems to monitor large gatherings from aerial viewpoints and cover areas that are difficult to monitor using fixed CCTV cameras.

The project can also be improved using Edge AI and Edge Computing technologies, where video processing and AI computations are performed locally near the cameras instead of relying entirely on cloud servers. This can reduce network latency, improve processing speed, and ensure faster emergency response during critical situations.

Integration with IoT (Internet of Things) sensors can further enhance the system by collecting environmental data such as temperature, humidity, air quality, noise levels, and crowd pressure information. This additional data can help authorities make better safety decisions during large public events.

More advanced deep learning models can be developed to improve crowd behavior analysis, anomaly detection, and crowd surge prediction accuracy. Predictive analytics can help identify dangerous situations such as stampedes, panic movements, or abnormal crowd behavior before they occur.

In future implementations, the system can support real-time facial recognition technologies for faster and more accurate missing person identification, significantly improving rescue and recovery operations during massive public gatherings. Additionally, incorporating privacy-preserving AI models will ensure data security and compliance with government regulations, making Trinetra ready for deployment in smart city infrastructure.

10.3 References

1. Afnan A. Shah (2024). “Enhancing Hajj and Umrah Rituals and Crowd Management Through AI Technologies: A Comprehensive Survey of Applications and Future Directions.” *IEEE Access*. Available online: <https://ieeexplore.ieee.org/document/10422392>
2. Zhou, B., Tang, X., & Wang, X. (2017). “Learning Collective Crowd Behaviors with Dynamic Pedestrian-Agents.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Available online: <https://ieeexplore.ieee.org/document/7458473>
3. Li, M., Zhang, D., & Chen, J. (2021). “Deep Learning for Crowd Counting: A Survey.” *IEEE Transactions on Neural Networks and Learning Systems*. Available online: <https://ieeexplore.ieee.org/document/9208951>
4. J. Anvar Shathik, Anil Saroliya, Gadala Suhasini, Sonal Borase, N. Noor Alleema, Namitha A.R. (2025). “Smart Vision Systems for Public Safety: Real-Time Crowd Monitoring and Anomaly Detection in Urban Spaces Using Deep Learning and Edge Computing.” *IJAM*. Available online: <https://www.ijamtes.org/gallery/146-june.pdf>